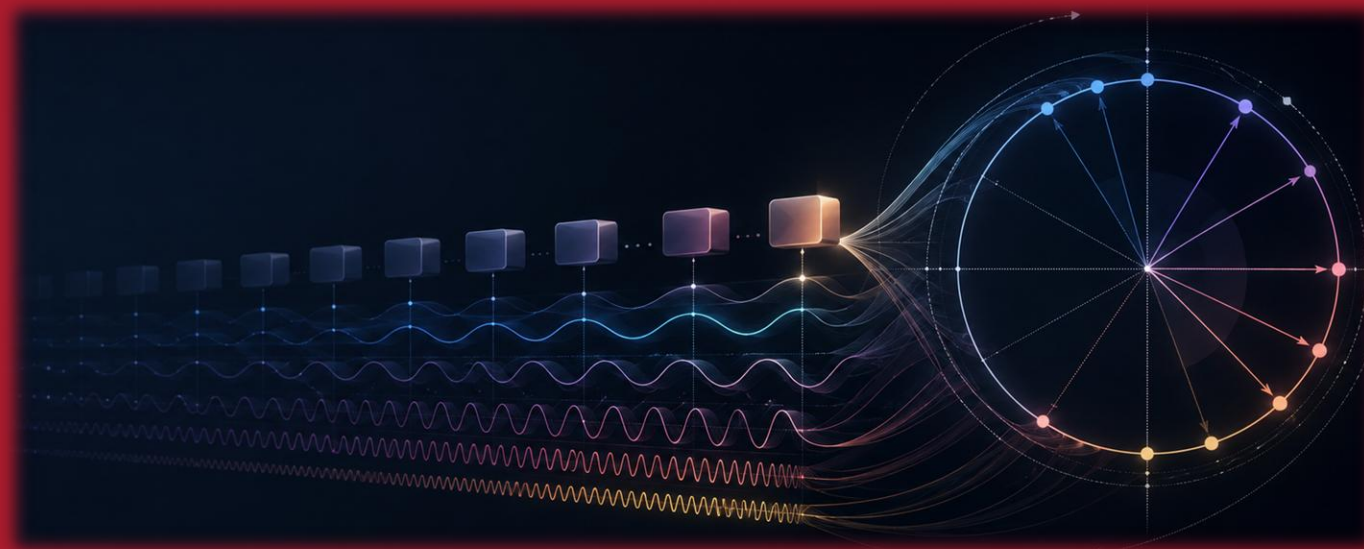


CS5326 – Advanced GenAI and Agents
Fall 2026

Agha Ali Raza

Position and Ordering



Position and ordering

- **Sequence of words:**
 - The dog hit the cat with a bat while favoring the rat
- Sequence of words with ordering and positional information:
 - The₁ dog₂ hit₃ the₄ cat₅ with₆ a₇ bat₈ while₉ favoring₁₀ the₁₁ rat₁₂
- But why did we suddenly start caring about the positions as soon as we started with the transformers?

Why didn't we discuss positional information in RNNs?

0

RNNs do not encode positional information in sequences and it is one of their major limitations.

0%

RNNs explicitly encode positional information in sequences.

0%

RNNs implicitly encode positional information in sequences.

0%

Sequence processing through RNNs is position agnostic.

0%

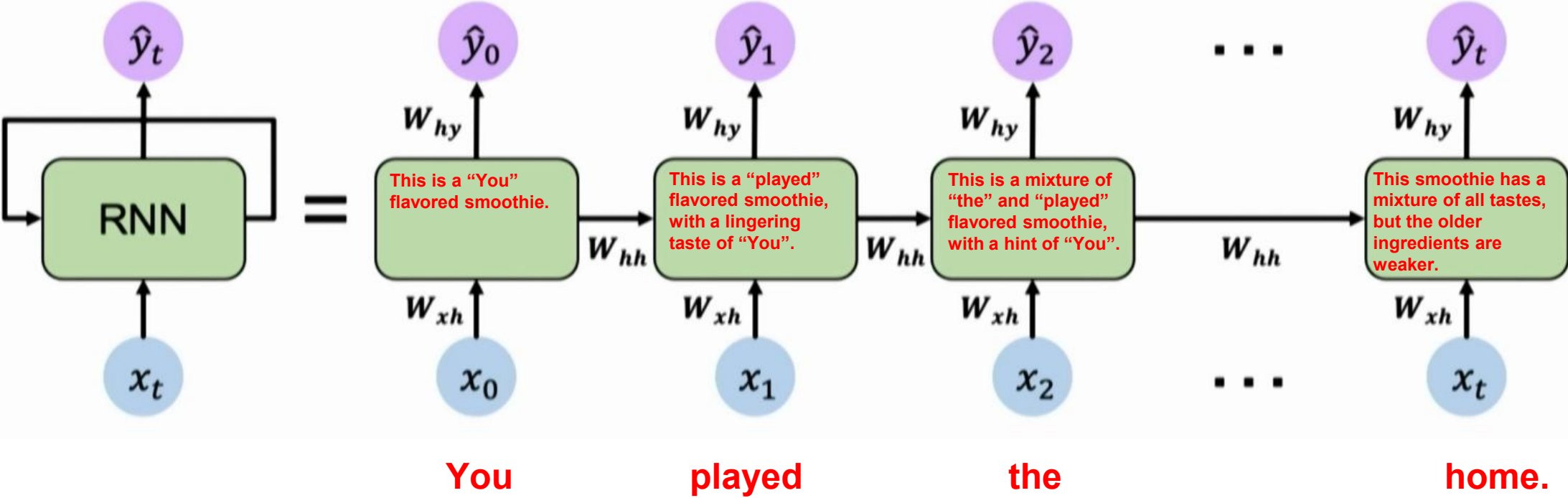
I have no idea what is going on right now!

0%

Computational Graph across Time



Re-use the **same weight matrices** at every time step



Positional Information in RNNs and LSTMs

RNNs and LSTMs did **not need an explicit positional encoding** in the Transformer sense because **order is built into the computation itself**. For a vanilla RNN,

$$h_t = f(x_t, h_{t-1})$$

so token x_t cannot be processed until the model has already processed $x_1, \dots, x_{t-1}$. The hidden state therefore acts as a running summary of everything seen **in that particular order**.

For example, compare:

- dog → bites → man

with

- man → bites → dog

For the first sequence,

- $h_1 = f(x_{\text{dog}}, h_0)$
- $h_2 = f(x_{\text{bites}}, h_1)$
- $h_3 = f(x_{\text{man}}, h_2)$.

For the second,

- $h'_1 = f(x_{\text{man}}, h_0)$
- $h'_2 = f(x_{\text{bites}}, h'_1)$
- $h'_3 = f(x_{\text{dog}}, h'_2)$.

Even though they contain exactly the same three words, generally

$$h_3 \neq h'_3$$

because changing the order changes the chain of computations.

Where does "position" actually live?

Consider the word **cat** appearing at different points:

- $x_1 = \text{cat}$

versus

- $x_{20} = \text{cat}.$

At $t = 1$,

- $h_1 = f(x_{\text{cat}}, h_0).$

At $t = 20$,

- $h_{20} = f(x_{\text{cat}}, h_{19}).$

The **input embedding** for "cat" may be exactly the same:

$$x_1 = x_{20}$$

but its contextual representation is not:

$$h_1 \neq h_{20}$$

because h_{19} contains information accumulated through nineteen previous recurrent steps.

So the RNN **doesn't explicitly say**:

- "this token is at position 20."

Instead, position is **implicitly reflected in the computational history**.

You can think of it as:

$$h_t = f(x_t, f(x_{t-1}, f(x_{t-2}, \dots f(x_1, h_0))))$$

Notice that $x_1, x_2, \dots, x_t$ are structurally not interchangeable. Their locations in this nested computation are different.

The LSTM

At every timestep we have an input vector X_t , previous hidden state H_{t-1} , previous cell state C_{t-1} , and LSTM computes the next hidden state H_t , and next cell state C_t at timestep t as follows:

$$F_t = \sigma(U_f H_{t-1} + W_f X_t + b_f)$$
$$[d \times 1] = \sigma([d \times d] \cdot [d \times 1] + [d \times V] \cdot [V \times 1]) + [d \times 1]$$

$$I_t = \sigma(U_i H_{t-1} + W_i X_t + b_i)$$
$$[d \times 1] = \sigma([d \times d] \cdot [d \times 1] + [d \times V] \cdot [V \times 1]) + [d \times 1]$$

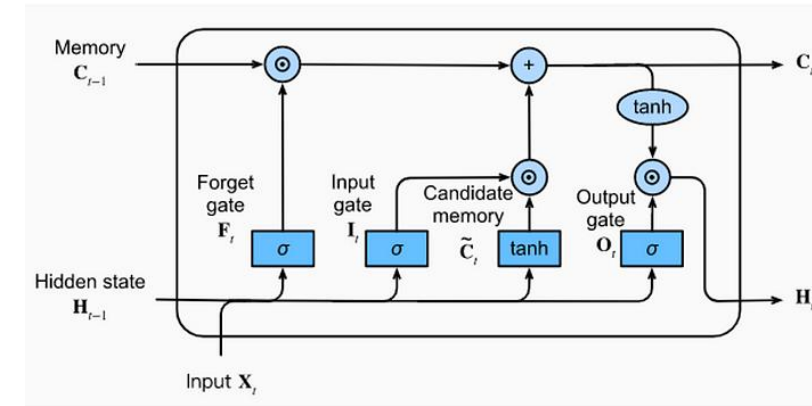
$$O_t = \sigma(U_o H_{t-1} + W_o X_t + b_o)$$
$$[d \times 1] = \sigma([d \times d] \cdot [d \times 1] + [d \times V] \cdot [V \times 1]) + [d \times 1]$$

$$\tilde{C}_t = \tanh(U_g H_{t-1} + W_g X_t + b_g)$$
$$[d \times 1] = \tanh([d \times d] \cdot [d \times 1] + [d \times V] \cdot [V \times 1]) + [d \times 1]$$

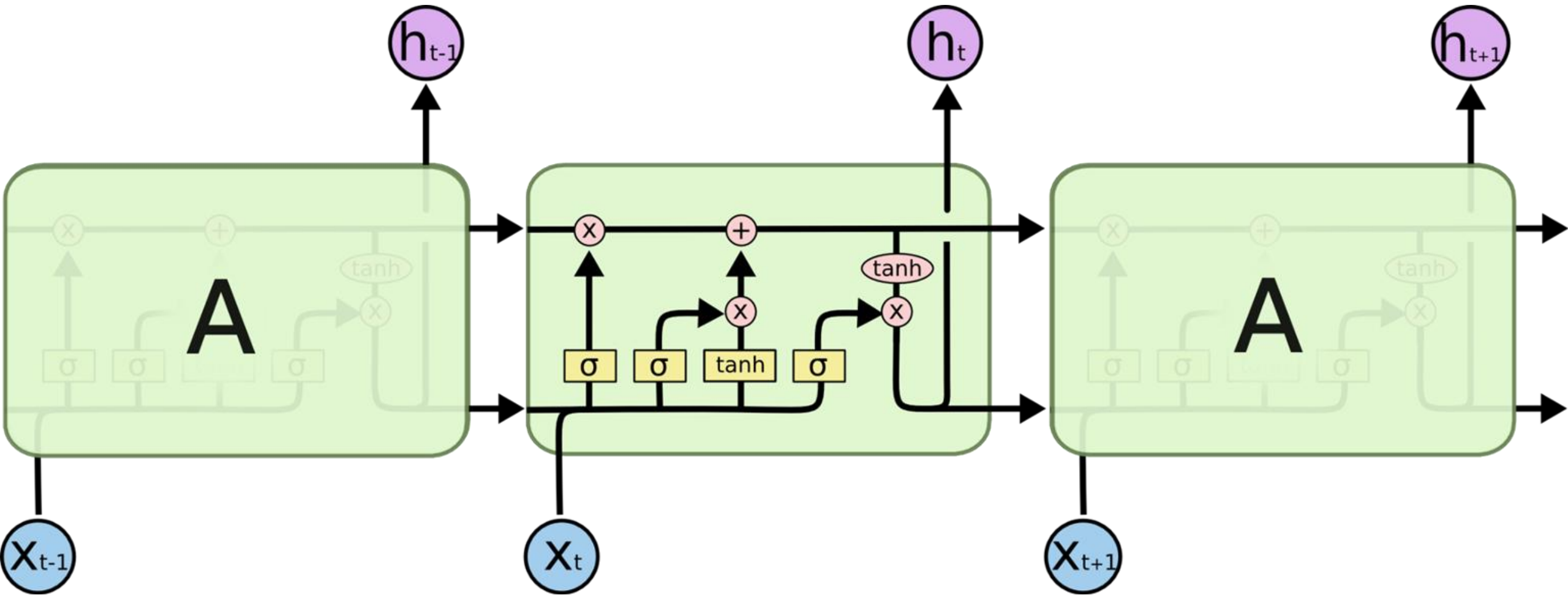
$$C_t = (F_t \odot C_{t-1}) + (I_t \odot \tilde{C}_t)$$
$$[d \times 1] = ([d \times 1] \odot [d \times 1]) + ([d \times 1] \odot [d \times 1])$$

$$H_t = O_t \odot \tanh(C_t)$$
$$[d \times 1] = [d \times 1] \odot \tanh([d \times 1])$$

$$\hat{Y}_t = \phi_o(W_{hy} H_t + b_y)$$
$$[V \times 1] = [V \times d] \cdot [d \times 1] + [V \times 1]$$



LSTM



LSTMs have a better memory mechanism

The important part for our discussion is still:

C_t depends on C_{t-1}

and

H_t depends on H_{t-1} .

So an LSTM also inherently knows sequence order through recurrence.

Ordering vs Absolute position

An RNN gets **ordering information inherently**, but it does not necessarily maintain a clean explicit representation such as

- $t = 37$.

It knows, through its state, something more like:

- "I arrived at this token after processing this sequence of preceding tokens."

Those are not quite the same thing.

An RNN *could learn* to count positions. For example, theoretically one hidden dimension could evolve roughly as

$$h_t^{(j)} = h_{t-1}^{(j)} + 1,$$

giving

$$0, 1, 2, 3, \dots$$

but standard RNNs are not explicitly required to do this.

So in RNN/LSTM: **order is intrinsic; position is implicit.**

Why does the Transformer have a problem that the RNN doesn't?

This becomes very clear if we compare the computations.

An RNN computes

$$x_1 \rightarrow h_1 \rightarrow x_2 \rightarrow h_2 \rightarrow x_3 \rightarrow h_3.$$

There is an arrow of time.

But vanilla self-attention receives

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

and computes

$$Q = XW_Q, K = XW_K, V = XW_V$$

followed by

$$\text{Attention}(X) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Nothing in these operations inherently says:

- x_1 came before x_2 .

If I permute the input tokens, self-attention simply permutes its outputs correspondingly.

- This property is called **permutation equivariance**.

Why Order Becomes Invisible

The word “dog” gets the same embedding vector whether it appears at position 1 or position 3.

So the attention score between “dog” and “bites” is identical in both sentences.

- In “Dog bites man”:

$$\text{score}(\text{dog}, \text{bites}) = (W_q \cdot e_{\text{dog}}) \cdot (W_k \cdot e_{\text{bites}})$$

- In “Man bites dog”:

$$\text{score}(\text{dog}, \text{bites}) = (W_q \cdot e_{\text{dog}}) \cdot (W_k \cdot e_{\text{bites}})$$

- Identical. This holds for every pair of tokens.

The full attention matrix for “Dog bites man” is identical to the full attention matrix for “Man bites dog.” Every value, every row, every column will be same.

"Dog bites man"				"Man bites dog"			
	Dog	bites	man		Man	bites	dog
Dog	0.34	0.41	0.25	Man	0.34	0.44	0.22
bites	0.30	0.38	0.32	bites	0.32	0.38	0.30
man	0.22	0.44	0.34	dog	0.25	0.41	0.34

Why Order Becomes Invisible

"Dog bites man"				"Man bites dog"			
	Dog	bites	man		Man	bites	dog
Dog	0.34	0.41	0.25	Man	0.34	0.44	0.22
bites	0.30	0.38	0.32	bites	0.32	0.38	0.30
man	0.22	0.44	0.34	dog	0.25	0.41	0.34

Shuffle the tokens in any order and the attention scores do not change.

- “Dog bites man” and “Man bites dog” and “Bites dog man” all produce the same attention pattern.

Embeddings are looked up by **token identity, not by position**.

- Position simply does not exist in the computation.

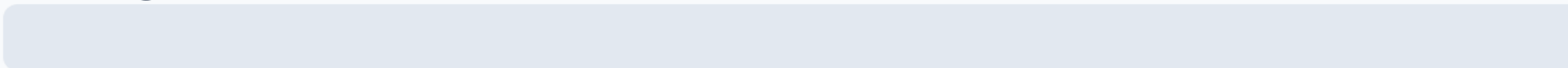
Without positional information, full self-attention treats the sequence as a set of tokens: it knows what tokens are present and how their contents relate, but not the order in which they occur.

- This is the problem that positional encodings exist to solve

Permutation Equivariance vs. Permutation Invariance

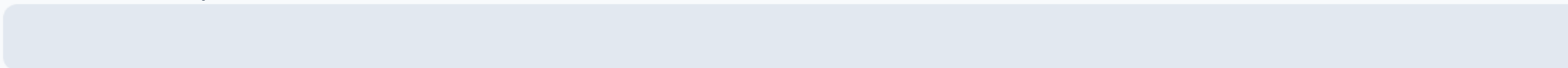


Same thing



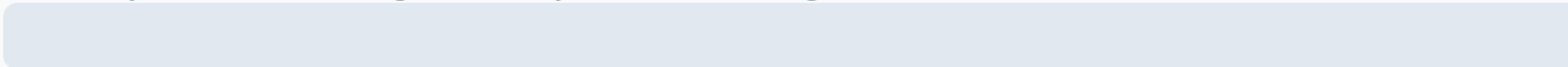
0%

Different concepts



0%

Absolutely, no idea. Who am I again? And why am I in this room right now?



0%

Permutation Equivariance

In simple English:

- If you shuffle the input tokens, vanilla self-attention processes the same tokens with the same relationships, just shuffled accordingly. It has no inherent knowledge of which token originally occupied position 1, 2, 3, etc.

For example, without positional information:

- [dog, bites, man]

versus

- [man, bites, dog]

contain the same embeddings. Self-attention alone has no independent positional signal telling it which ordering was intended.

Hence Transformers need something extra:

token information + position information.

The original Transformer used

$$x_t + PE_t$$

Permutation, Invariance vs Equivariance

Invariance: change the input ordering, and the output stays the same.

$$f(PX) = f(X)$$

Example: summing a set.

$$f([2, 5, 7]) = 14$$

and

$$f([7, 2, 5]) = 14$$

So summation is **permutation-invariant**.

Equivariance: change the input ordering, and the output changes in the same corresponding way.

$$f(PX) = P f(X)$$

Example: suppose a layer maps

$$[A, B, C] \rightarrow [a, b, c]$$

If we permute the input:

$$[C, A, B]$$

the output becomes

$$[c, a, b]$$

So the output is not unchanged; it is **permuted consistently with the input**.

- In one line:

Invariant: input changes, output stays the same

Equivariant: input changes, output changes correspondingly

Vanilla self-attention without positional information is **permutation-equivariant**, not permutation-invariant.

What is P

P is a **permutation operator** (often represented as a permutation matrix). It simply reorders elements.

For example, if

$$X = \begin{bmatrix} A \\ B \\ C \end{bmatrix}$$

and we want to reorder it as

$$[C, A, B]$$

we can use

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Then

$$PX = \begin{bmatrix} C \\ A \\ B \end{bmatrix}.$$

So in

$$f(PX) = Pf(X),$$

the meaning is:

If I shuffle the inputs using P , the outputs get shuffled in exactly the same way.

- That is **permutation equivariance**.

Equivariance vs. Invariance

PERMUTATION EQUIVARIANCE

Reorder the input...

[dog, bites, man]



[man, bites, dog]

...and the output is reordered in exactly the same way.

equivariant: $f(PX) = P f(X)$

PERMUTATION INVARIANCE

Reorder the input...

[dog, bites, man]



[man, bites, dog]

...but one pooled output remains unchanged.

invariant: $g(PX) = g(X)$

Wait! Did we just learn
that RNNs and LSTMs
were better than
Transformers?

And that we will spend
two lectures solving
what was already
solved with RNNs?



There was a price for the RNN's "free" ordering information

The RNN got order naturally because it computed sequentially:

$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots \rightarrow h_T.$$

But that same feature was also one of its major disadvantages.

You cannot compute h_{100} until you've computed

$$h_1, h_2, \dots, h_{99}.$$

So training has an inherently sequential dependency.

For a Transformer, all token representations can initially be processed in parallel:

$$x_1, x_2, x_3, \dots, x_T.$$

That's enormously advantageous computationally.

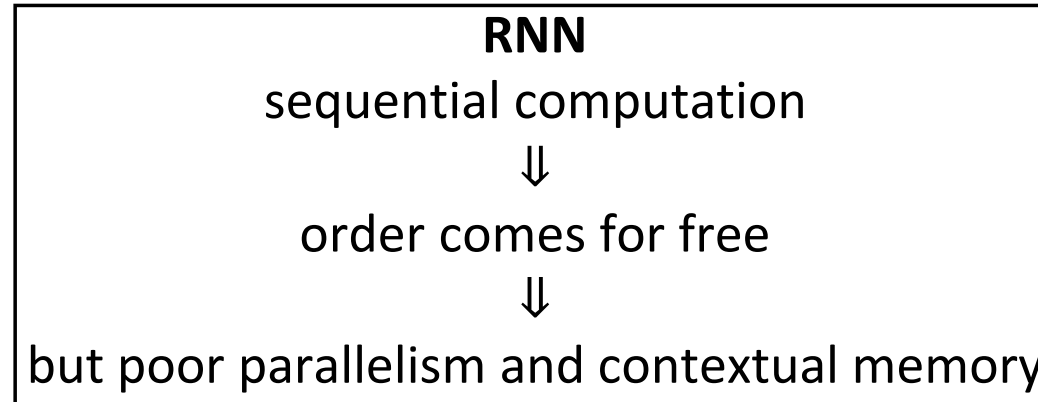
And

Do not forget that RNNs rapidly “forget” context due to the same “advantage”!

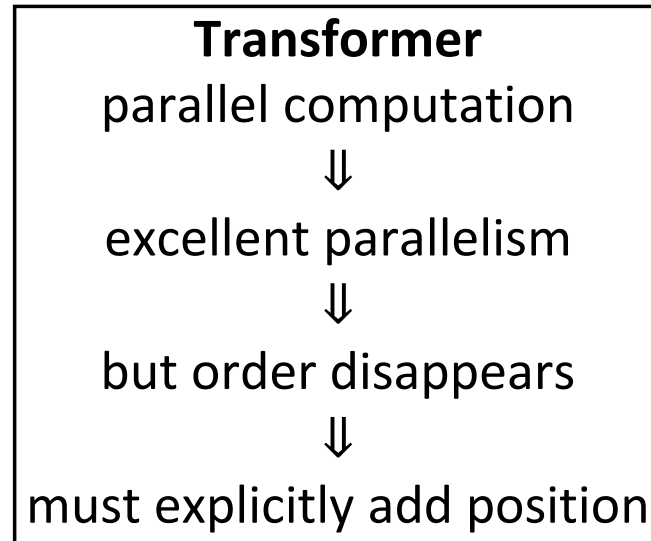
And, LSTMs came to save the day but were not quite successful at it.

There was a price for the RNN's "free" ordering information

- So, somewhat beautifully, the story is:



versus



That is probably the most important conceptual bridge from **RNN/LSTM** → **Transformer positional encoding**.

Problem Summary and Proposed Solution

For language, **changing order can change meaning**, but positionless self-attention cannot make the token representations reflect that change. Take:

$$S_1 = [\text{dog}, \text{bites}, \text{man}] \text{ and } S_2 = [\text{man}, \text{bites}, \text{dog}].$$

Suppose positionless self-attention gives

$$f(S_1) = [h_{\text{dog}}, h_{\text{bites}}, h_{\text{man}}].$$

Since $S_2 = PS_1$, equivariance guarantees

$$f(S_2) = Pf(S_1) = [h_{\text{man}}, h_{\text{bites}}, h_{\text{dog}}].$$

Here's the problem:

h_{dog} is exactly the same in both sentences

and similarly,

h_{man} is exactly the same in both sentences.

Yet linguistically:

- **dog bites man**: dog = agent, man = patient
- **man bites dog**: man = agent, dog = patient

The model has no way to encode that difference because the only thing that changed was **where the words occurred**, and it has no positional signal.

So the bad part is **not equivariance itself**. The bad part is:

representation of a token cannot depend on its position/order

without some positional mechanism.

Problem Summary and Proposed Solution

If you then do sequence classification with permutation-invariant pooling, it gets even worse:

$$g(f(S_1)) = g(f(S_2)),$$

so the two sentences become literally indistinguishable.

That is why we propose to inject position:

$$x_i \rightarrow x_i + PE_i.$$

Now moving "dog" from position 1 to position 3 changes its input representation:

$$x_{\text{dog}} + PE_1 \neq x_{\text{dog}} + PE_3,$$

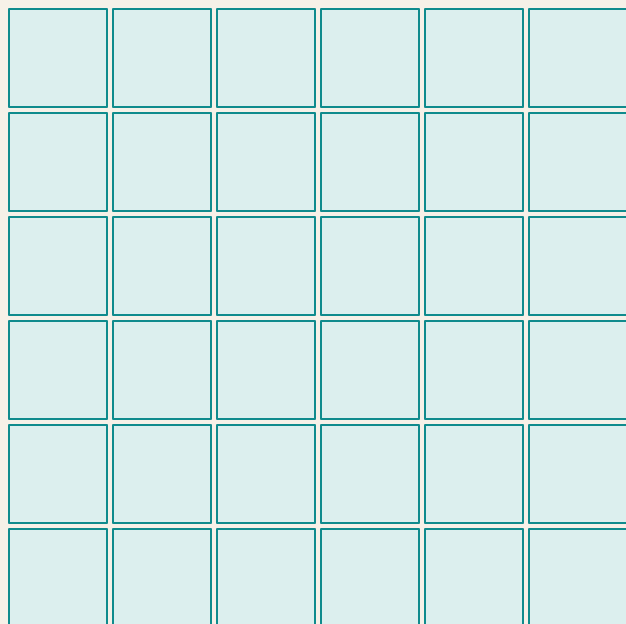
allowing self-attention to learn that

dog before bites $\neq$ dog after bites.

That is the actual problem positional encoding solves.

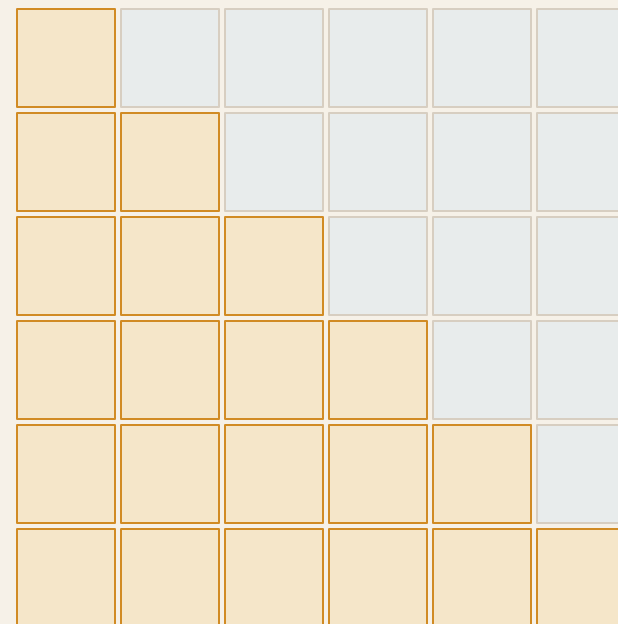
Note: A causal mask adds asymmetry, not distance

UNMASKED SELF-ATTENTION



Every token can inspect every token.

CAUSAL SELF-ATTENTION



A token can inspect only its prefix.

The mask says earlier/later; it does not explicitly encode "7 tokens apart."

Positional Encoding

Simplest Solution!

- Be honest, what's the simplest idea that comes to mind when we think of adding positional information to the embedding vectors?
- **Adding Raw Position Numbers (0, 1, 2, 3, 4, 5,...)**
 - The Transformer needs to know where each token is. The simplest idea is to just tell it directly by adding its position.
 - Take the position of each token as an integer and add it to the embedding.
 - Token at position 0 gets +0. Token at position 1 gets +1. Token at position 511 gets +511.

This (position 0)	quick (position 1)		fox (position 100)	dog (position 500)																
embedding (e)	embedding (e)		embedding (e)	embedding (e)																
<table><tr><td>0.32</td></tr><tr><td>-0.71</td></tr><tr><td>0.15</td></tr><tr><td>0.58</td></tr></table>	0.32	-0.71	0.15	0.58	<table><tr><td>0.32</td></tr><tr><td>-0.71</td></tr><tr><td>0.15</td></tr><tr><td>0.58</td></tr></table>	0.32	-0.71	0.15	0.58		<table><tr><td>0.32</td></tr><tr><td>-0.71</td></tr><tr><td>0.15</td></tr><tr><td>0.58</td></tr></table>	0.32	-0.71	0.15	0.58	<table><tr><td>0.32</td></tr><tr><td>-0.71</td></tr><tr><td>0.15</td></tr><tr><td>0.58</td></tr></table>	0.32	-0.71	0.15	0.58
0.32																				
-0.71																				
0.15																				
0.58																				
0.32																				
-0.71																				
0.15																				
0.58																				
0.32																				
-0.71																				
0.15																				
0.58																				
0.32																				
-0.71																				
0.15																				
0.58																				
+	+		+	+																
position vector (pos)	position vector (pos)		position vector (pos)	position vector (pos)																
<table><tr><td>0.00</td></tr><tr><td>0.00</td></tr><tr><td>0.00</td></tr><tr><td>0.00</td></tr></table>	0.00	0.00	0.00	0.00	<table><tr><td>1.00</td></tr><tr><td>1.00</td></tr><tr><td>1.00</td></tr><tr><td>1.00</td></tr></table>	1.00	1.00	1.00	1.00		<table><tr><td>100.00</td></tr><tr><td>100.00</td></tr><tr><td>100.00</td></tr><tr><td>100.00</td></tr></table>	100.00	100.00	100.00	100.00	<table><tr><td>500.00</td></tr><tr><td>500.00</td></tr><tr><td>500.00</td></tr><tr><td>500.00</td></tr></table>	500.00	500.00	500.00	500.00
0.00																				
0.00																				
0.00																				
0.00																				
1.00																				
1.00																				
1.00																				
1.00																				
100.00																				
100.00																				
100.00																				
100.00																				
500.00																				
500.00																				
500.00																				
500.00																				
=	=		=	=																
result e' = e + pos	result e' = e + pos		result e' = e + pos	result e' = e + pos																
<table><tr><td>0.32</td></tr><tr><td>-0.71</td></tr><tr><td>0.15</td></tr><tr><td>0.58</td></tr></table>	0.32	-0.71	0.15	0.58	<table><tr><td>1.32</td></tr><tr><td>0.29</td></tr><tr><td>1.15</td></tr><tr><td>1.58</td></tr></table>	1.32	0.29	1.15	1.58		<table><tr><td>100.32</td></tr><tr><td>99.29</td></tr><tr><td>100.15</td></tr><tr><td>100.58</td></tr></table>	100.32	99.29	100.15	100.58	<table><tr><td>500.32</td></tr><tr><td>499.29</td></tr><tr><td>500.15</td></tr><tr><td>500.58</td></tr></table>	500.32	499.29	500.15	500.58
0.32																				
-0.71																				
0.15																				
0.58																				
1.32																				
0.29																				
1.15																				
1.58																				
100.32																				
99.29																				
100.15																				
100.58																				
500.32																				
499.29																				
500.15																				
500.58																				

Raw Position Numbers

Every token embedding is a vector of numbers, typically in the range of -1 to +1.

The proposal is straightforward: take the position index and add it as a scalar to every dimension of the embedding vector.

$$e_i' = e_i + i$$

where i is the position of the token in the sequence.

For a sequence “Dog bites man”:

- Position 0: $e_{\text{dog}}' = e_{\text{dog}} + 0$
- Position 1: $e_{\text{bites}}' = e_{\text{bites}} + 1$
- Position 2: $e_{\text{man}}' = e_{\text{man}} + 2$

Now “Dog bites man” and “Man bites dog” produce different embeddings because the same word gets a different number added depending on where it appears.

But we will face some issues with this approach.

- **Your turn!**

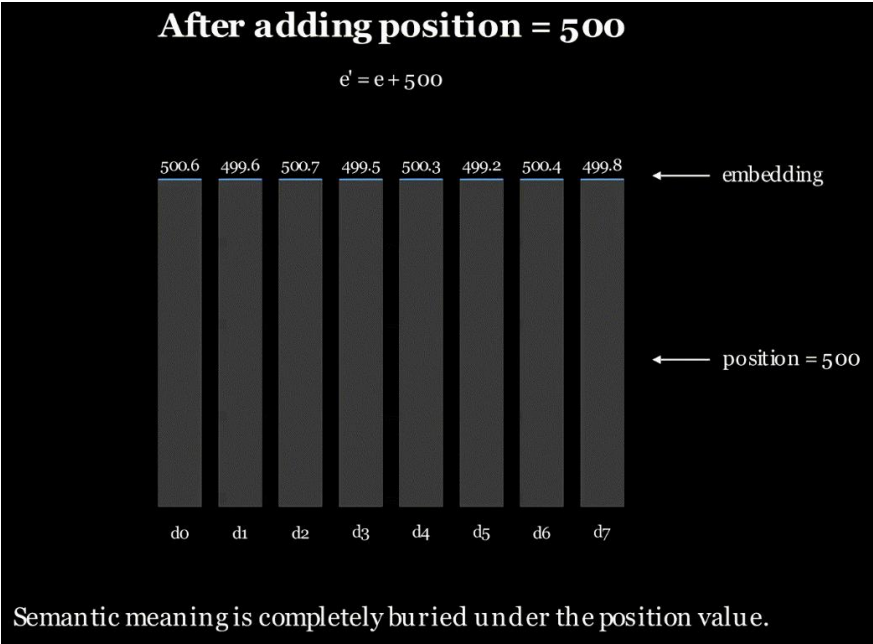
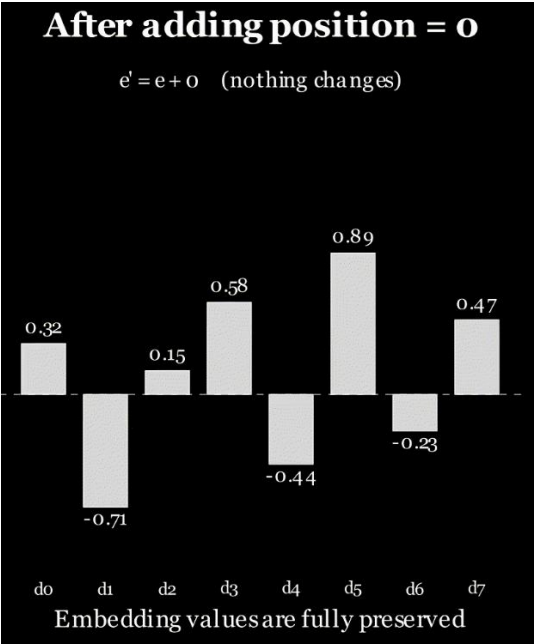
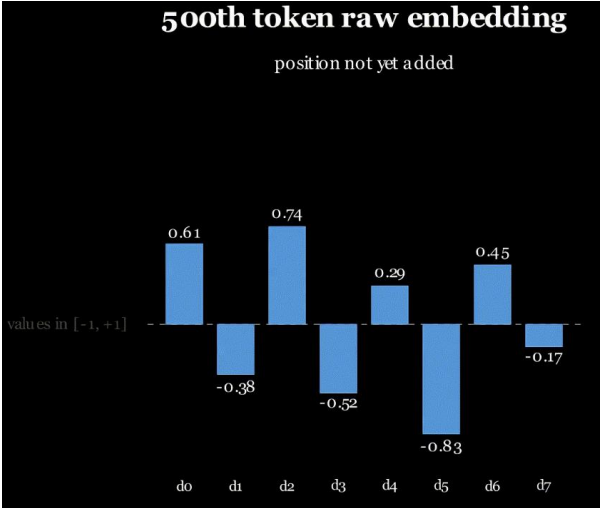
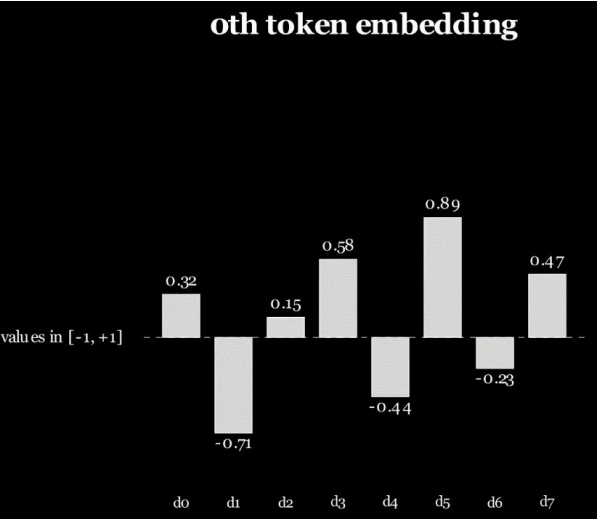
The Scale Problem

- Embedding values typically live in the range of -1 to +1. These are small, carefully learned numbers that encode the meaning of each token.
 - But the range of values can vary relative to a higher number.
- Position 500:
 - We add 500 to every dimension of the embedding.
 - A dimension that was 0.3 becomes 500.3.
 - A dimension that was -0.7 becomes 499.3.
- The positional number completely dominates the embedding.
- The semantic content of the token is overshadowed by a massive positional value.
 - The model can barely see the embedding of the token.

Differential Treatment:

- At position 0, the embedding is untouched.
- At position 500, the embedding is almost entirely overwritten by the position value.
- Tokens near the beginning of a sequence and tokens near the end live in completely different numerical ranges, not only because they mean different things, but also because of where they appear.

The Scale Problem



No Upper Bound

- This approach has no fixed range.
 - The position value grows without limit as the sequence gets longer.
- A model trained on sequences of length 512 has seen position values from 0 to 511.
 - At inference time, if the input has 1024 tokens, the model suddenly sees position values up to 1023.
 - It has never encountered numbers this large during training.
- This is an out-of-distribution problem.
- The model has no way to generalize to positions it has never seen.

Inconsistent Distance

- The absolute difference between position 1 and position 2 is 1.
 - The absolute difference between position 500 and position 501 is also 1.
- But relative to the **position values** themselves, these gaps are very different.
 - 1 to 2 is a 100 percent jump (doubles).
 - 500 to 501 is 0.2 percent jump ($1/500$).
 - "The next token" has no single meaning.
- The model cannot learn a consistent notion of “subsequent positions” because the same gap of absolute positional difference of 1 looks completely different depending on where in the sequence it occurs.
- **So, what’s the simplest solution that comes to mind?**

Normalize Positions into [0,1]

Raw integer positions failed because the values were too large. They overwhelmed the embeddings and had no upper bound.

Instead, can we just squeeze all position values into the range [0, 1]?

The Idea

Divide each position by the length of the sequence minus one.

$$PE(pos) = \frac{pos}{L - 1}$$

where L is the total number of tokens in the sequence.

For a sequence of length 512:

- Position 0 $\rightarrow$ 0.0
- Position 255 $\rightarrow$ 0.5
- Position 511 $\rightarrow$ 1.0

Every position now maps to a value between 0 and 1.

- No matter how long the sequence is, the values never exceed 1.
- They sit comfortably in the same range as the embedding values.
- The scale problem is gone.
- The model no longer has to deal with position values like 500 drowning out embedding values like 0.3.

Any Problems?

The Spacing Problem

Consider two sequences of different lengths.

- A short sequence with 10 tokens:

[0.0, 0.11, 0.22, 0.33, 0.44, 0.56, 0.67, 0.78, 0.89, 1.0]

The spacing between consecutive positions is 0.11.

- A long sequence with 1000 tokens:

[0.0, 0.001, 0.002, 0.003, ..., 0.999, 1.0]

The spacing between consecutive positions is 0.001.

The gap between adjacent tokens is 100 times smaller in the long sequence than in the short sequence.

Two tokens that are “one step apart” look very different to the model depending on sequence length.

Same Position, Different Values

The same position index maps to completely different values depending on the sequence length.

- Position 5 in a 10 token sequence:

$$PE(5) = \frac{5}{9} = 0.556$$

- Position 5 in a 1000 token sequence:

$$PE(5) = \frac{5}{999} = 0.005$$

- The fifth token gets the value 0.556 in one case and 0.005 in the other. These are not even close.
- The model cannot learn what “position 5” means because the value it receives changes with every input. A model trained mostly on short sequences will associate 0.5 with the middle of a sentence.
 - When it sees a long sequence where 0.5 maps to position 500, the learned association breaks.

Why This Is Fundamental

This scheme is **relative to sequence length**.

- It does not encode absolute position. It encodes “how far through the sequence are we.”
- Position 0 always means “beginning.”
- Position 1.0 always means “end.”
- But everything in between shifts depending on L .

This creates two failures:

- **No consistent position identity.** The same position index produces different values for different sequence lengths. The model cannot learn a stable representation for any position.
- **No consistent spacing.** The distance between consecutive positions depends on L . The model cannot learn a uniform notion of “adjacent tokens” because the numerical gap changes per sequence.
- **Now what?**

Scalars to Vectors

- Both attempts so far used a single number to represent each position.
 - The first attempt used numbers that were too large.
 - The second attempt used numbers that changed meaning depending on context length.
- What if instead of a single number, we represented each position as a vector?
 - And what if that vector used a fixed, length- i pattern that gave every position a unique and consistent representation?
- This is exactly what binary encoding do.

Binary Encodings

- Both attempts so far used a single number to represent each position.
- What if instead of a single number, we represented each position as a vector?
 - And what if that vector used a fixed, length- i pattern that gave every position a unique and consistent representation?
- This is exactly what binary encoding do.
 - Each position gets a unique vector of 0s and 1s.
 - The dimensionality of the vector is $\lceil \log_2(L) \rceil$, where L is the maximum sequence length.
 - For a sequence of up to 512 tokens, we need 9 bits.

Position	Binary Vector
0	[0, 0, 0, 0, 0, 0, 0, 0, 0]
1	[0, 0, 0, 0, 0, 0, 0, 0, 1]
2	[0, 0, 0, 0, 0, 0, 0, 1, 0]
5	[0, 0, 0, 0, 0, 0, 1, 0, 1]
255	[0, 1, 1, 1, 1, 1, 1, 1, 1]
511	[1, 1, 1, 1, 1, 1, 1, 1, 1]

What Binary Encoding Gets Right

This approach fixes every problem from the previous two attempts.

- **Bounded values.** Every entry in the vector is either 0 or 1. No position value ever exceeds 1. There is no risk of drowning out the embedding.
- **Unique per position.** Every integer has a distinct binary representation. No two positions share the same vector. Position 5 is always $[0, 0, 0, 0, 0, 0, 1, 0, 1]$, regardless of how long the sequence is.
- **Length independent.** Unlike normalized positions, the encoding of position 5 does not change when the sequence length changes. Position 5 is the same vector whether the sequence has 10 tokens or 10,000 tokens.
- **Fixed dimensionality.** The encoding uses $\log_2(n)$ dimensions. This grows very slowly. 10 bits can handle sequences up to 1024. 20 bits can handle sequences up to 1,048,576.

But something interesting is hidden in how these bits change across positions.

The Frequency Pattern in Binary

The binary representations for positions 0 through 7 is as follows, and if we look at each bit column separately.

Position	Bit 2 (2^2)	Bit 1 (2^1)	Bit 0 (2^0)
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Now read each column from top to bottom.

- **Bit 0** (the rightmost, least significant bit) flips every single position: 0, 1, 0, 1, 0, 1, 0, 1. It completes a **full cycle every 2 positions**.
- **Bit 1** flips every 2 positions: 0, 0, 1, 1, 0, 0, 1, 1. It completes a full cycle every 4 positions.
- **Bit 2** flips every 4 positions: 0, 0, 0, 0, 1, 1, 1, 1. It completes a full cycle every 8 positions.

Frequencies

- The frequency of that wave depends on which bit position it is.

Binary Encoding: Frequency Pattern (Positions 0–7)				
Position (decimal)	Bit 2 (2^2) Period = 8	Bit 1 (2^1) Period = 4	Bit 0 (2^0) Period = 2	Binary Vector [Bit 2, Bit 1, Bit 0]
0	0	0	0	[0 , 0 , 0]
1		0	1	[0 , 0 , 1]
2		1	0	[0 , 1 , 0]
3		1	1	[0 , 1 , 1]
4	1	0	0	[1 , 0 , 0]
5		0	1	[1 , 0 , 1]
6		1	0	[1 , 1 , 0]
7		1	1	[1 , 1 , 1]

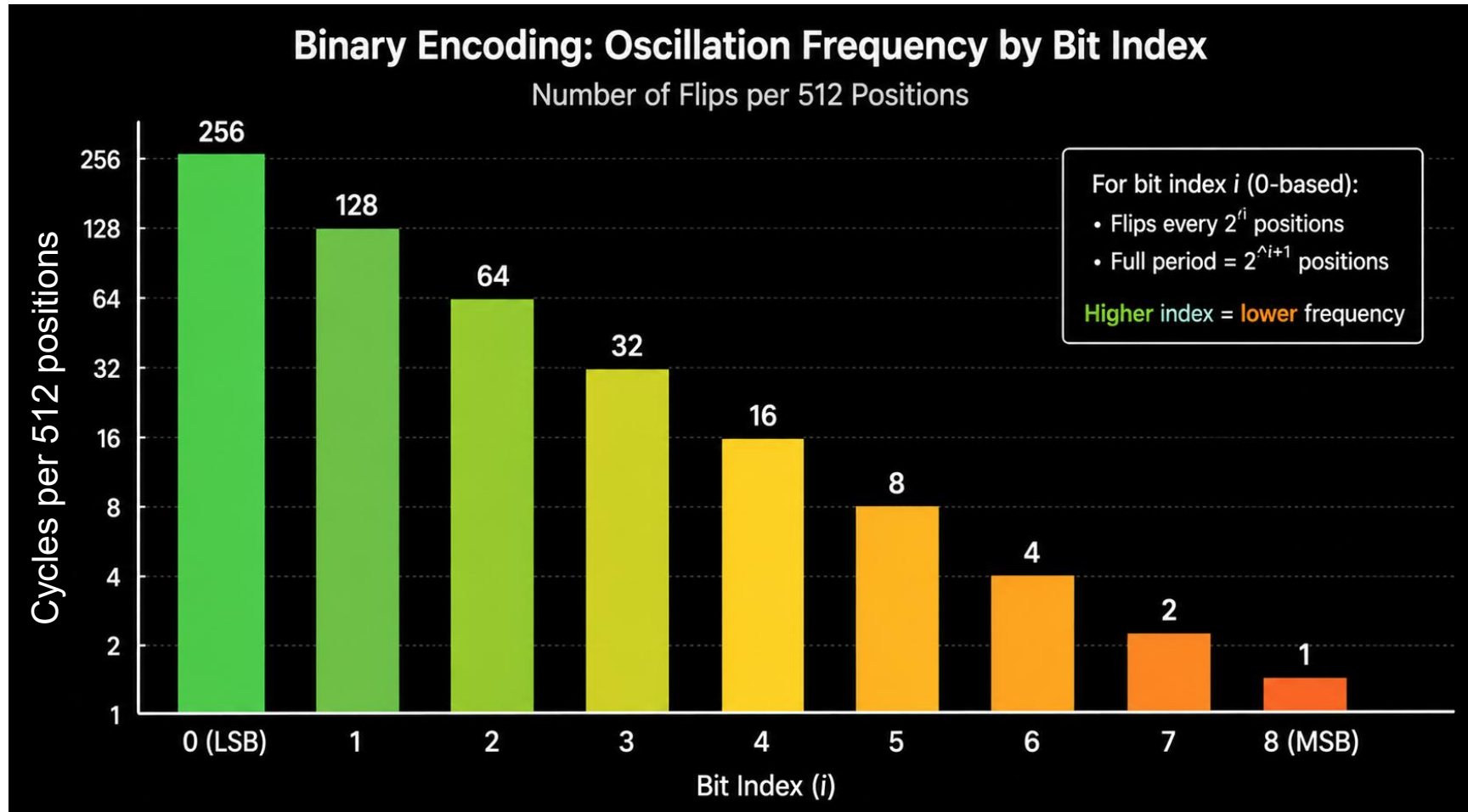
LSB vs MSB: Fast Bits and Slow Bits

- This pattern generalizes to any number of bits. For bit position i (counting from the right, starting at 0):
Oscillation period of bit $i = 2^{i+1}$ positions
- The **least significant bit** (LSB, rightmost, $i = 0$) oscillates the fastest. It flips at every single position. It has a period of 2.
- The **most significant bit** (MSB, leftmost, $i = d - 1$) oscillates the slowest. For a 9 bit encoding, it flips every 256 positions. It has a period of 512.
- Bits on the right change rapidly. Bits on the left change slowly. Each bit position captures positional information at a different scale.

Bit Position	Flips Every	Period	Role
Bit 0 (LSB)	1 position	2	Finest grain, changes constantly
Bit 1	2 positions	4	
Bit 2	4 positions	8	
Bit 3	8 positions	16	
...	...	...	
Bit 8 (MSB)	256 positions	512	Coarsest grain, barely changes

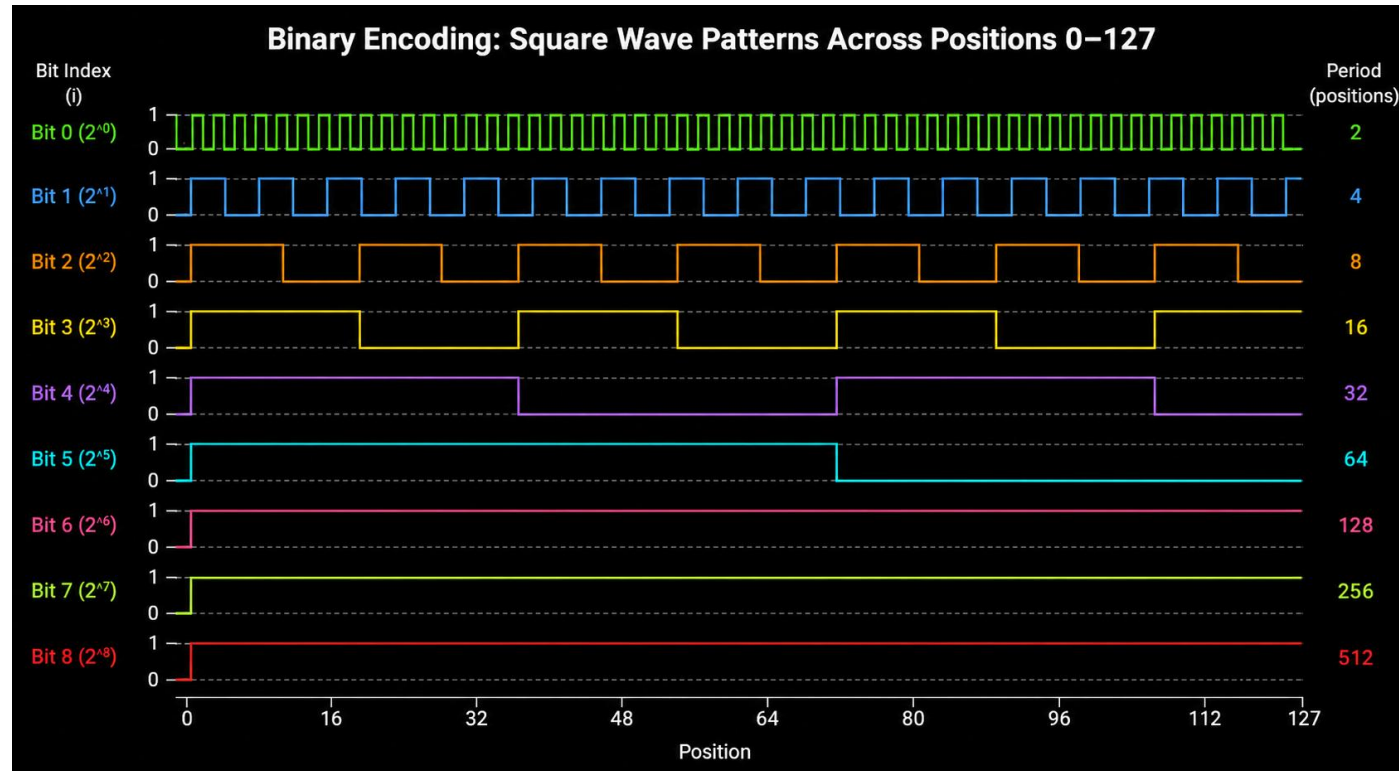
LSB vs MSB: Fast Bits and Slow Bits

- This pattern generalizes to any number of bits. For bit position i (counting from the right, starting at 0):
Oscillation period of bit $i = 2^{i+1}$ positions



Visualizing the Square Waves

- If you plot the value of each bit across all positions, you see a series of square waves stacked on top of each other. Each wave has exactly half the frequency of the one below it.



- This is a multi-frequency encoding.** The lowest bit gives fine-grained position information (is this an even or odd position?). The highest bit gives coarse position information (are we in the first half or second half of the sequence?).
- This multi frequency structure is the most important observation about binary encoding. It will directly motivate sinusoidal positional encoding.**

The Discontinuity Problem

Despite this awesome frequency structure, binary encoding has a flaw.

Look at positions 3 and 4:

- Position 3: [0, 1, 1]
- Position 4: [1, 0, 0]

These two positions are adjacent. They are one step apart. But their binary vectors differ in all three bits. The distance between them in vector space is large.

Now look at positions 2 and 3:

- Position 2: [0, 1, 0]
- Position 3: [0, 1, 1]

Also adjacent. Also one step apart. But only one bit differs. The distance between them is small.

Problem: Adjacent positions have wildly inconsistent distances in the encoding space.

- The transition from 3 to 4 is a large jump. The transition from 2 to 3 is a tiny step.
 - There is no smooth relationship between position and encoding.
- This happens because binary numbers carry over.
 - When all lower bits are 1, the next increment flips them all to 0 and flips the next higher bit to 1.
 - These carry overs cause sudden large changes in the vector for what should be a small step in position.

Why Discontinuity Matters

- Neural networks learn smooth functions. They work best when small changes in input produce small changes in output. If two positions are close together, their encodings should also be close together.
- Binary encoding violates this. The model cannot learn a smooth notion of “nearby positions” because the encoding jumps unpredictably between adjacent positions.

What We Keep, What We Fix

Binary encoding gave us two valuable ideas:

- **Multi-frequency structure.** Different bits capture position at different scales.
 - Fast bits for fine detail, slow bits for coarse structure.
- **Vector representation.** Each position is a vector, not a single number.

But it also has one critical issue:

- **Discrete jumps.** The square wave transitions between 0 and 1 are discontinuous. Adjacent positions can have very different encodings.

The fix is can be simple.

- Replace the square waves with smooth waves.
- Replace the discrete 0/1 flips with continuous sine and cosine functions.

Keep the multi-frequency structure. Make it smooth.

The structure is right; the waveform is wrong

keep the periods

remove the jumps

period 8



period 16



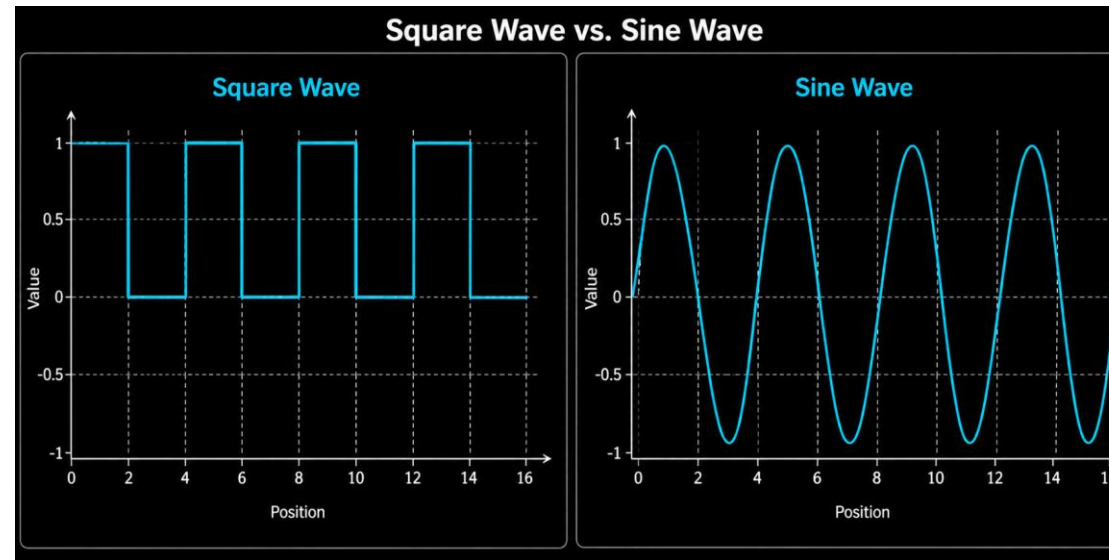
period 32



binary square waves

Sinusoidal Encoding

- The Smoothest Periodic Function
 - The smoothest possible periodic function is the sine wave or cosine wave.
- A sine wave does not jump.
 - It rises and falls continuously.
 - Adjacent points on the wave are always close to each other in value.
 - Two positions that are near each other will always produce sine values that are near each other.
- Compare a square wave and a sine wave at the same frequency:



- Both waves oscillate at the same rate. Both repeat at the same period. The difference is how they get from low to high. The square wave jumps. The sine wave moves smoothly.

Building a Multi-Frequency Sinusoidal Encoding

If we take inspiration from binary encoding, we want multiple sine waves stacked together. Each one oscillates at a different frequency.

- For position pos and dimension index i , the most natural starting point is:
$$PE(pos, i) = \sin(pos \cdot \omega_i)$$

Where ω_i is the frequency for dimension i .

Different dimensions get different frequencies.

- Some dimensions oscillate fast, like the LSB in binary.
- Others oscillate slowly, like the MSB.

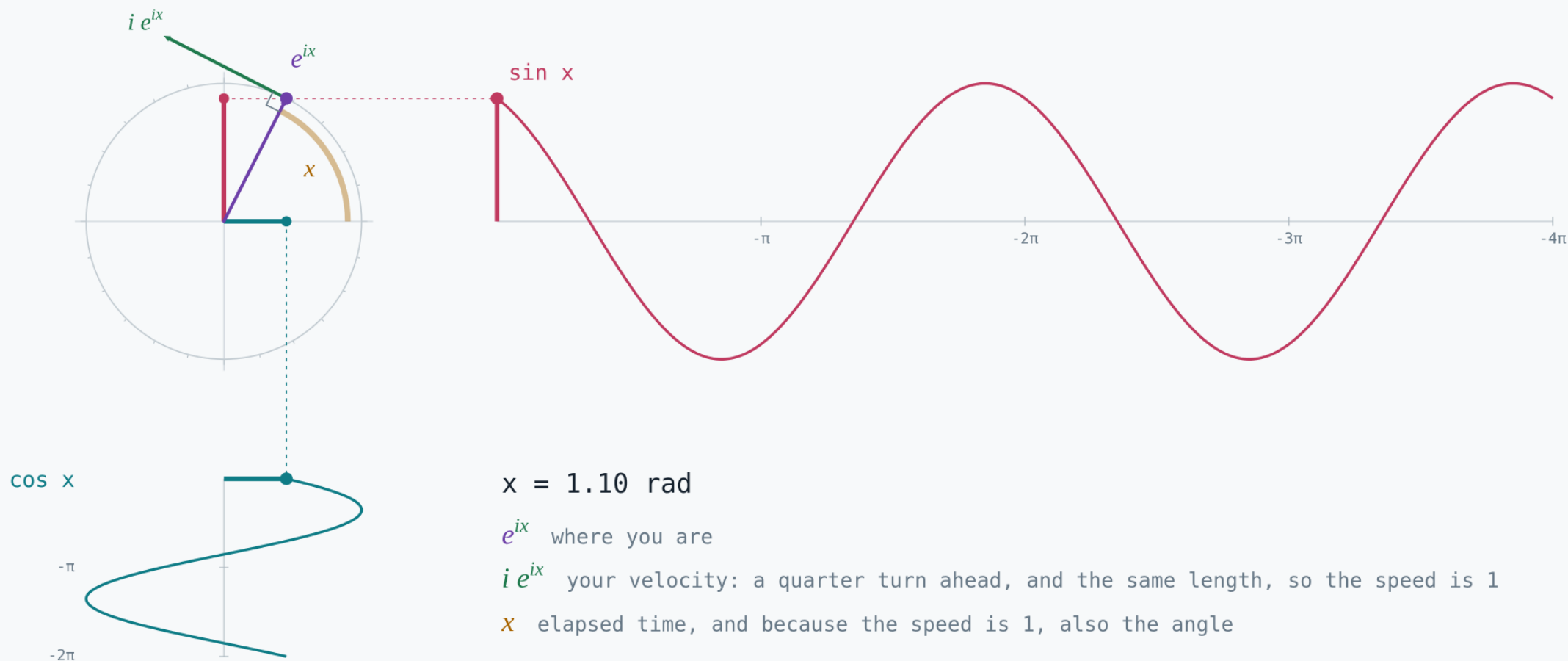
We now have to answer three questions:

1. How does this scheme work?
2. How do we choose the frequencies ω_i for each dimension?
3. Is using only sine enough, or do we also need cosine?

The original Transformer paper answers both. Each one solves a specific problem.

Growth at a right angle is a circle

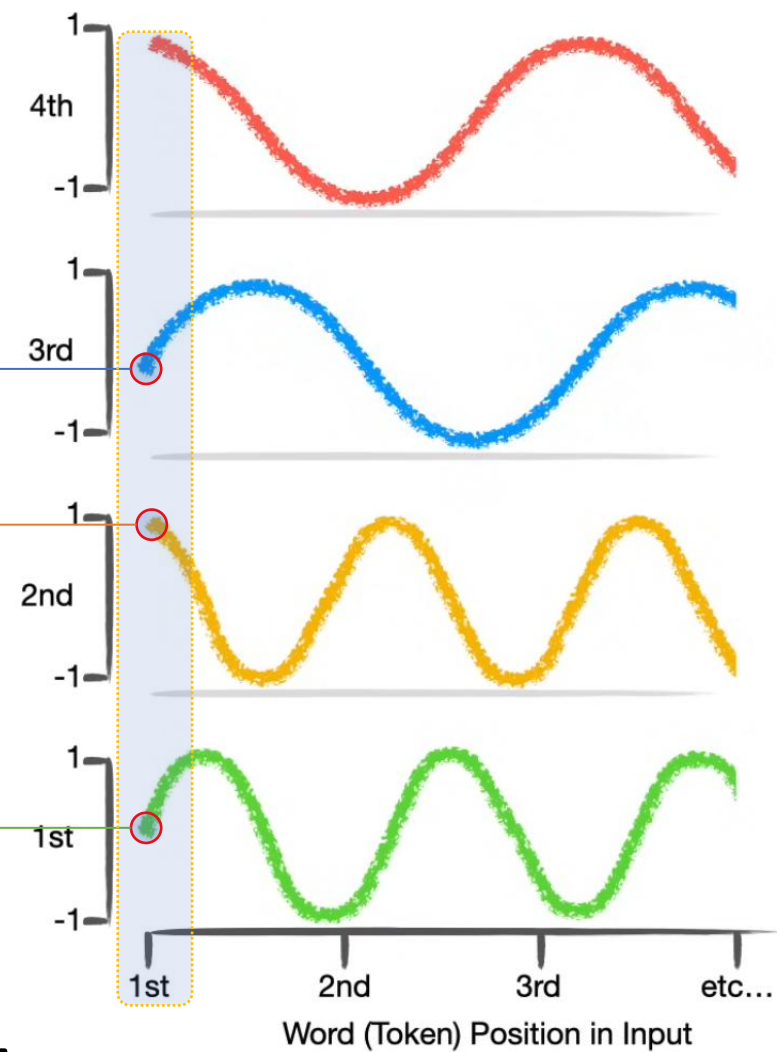
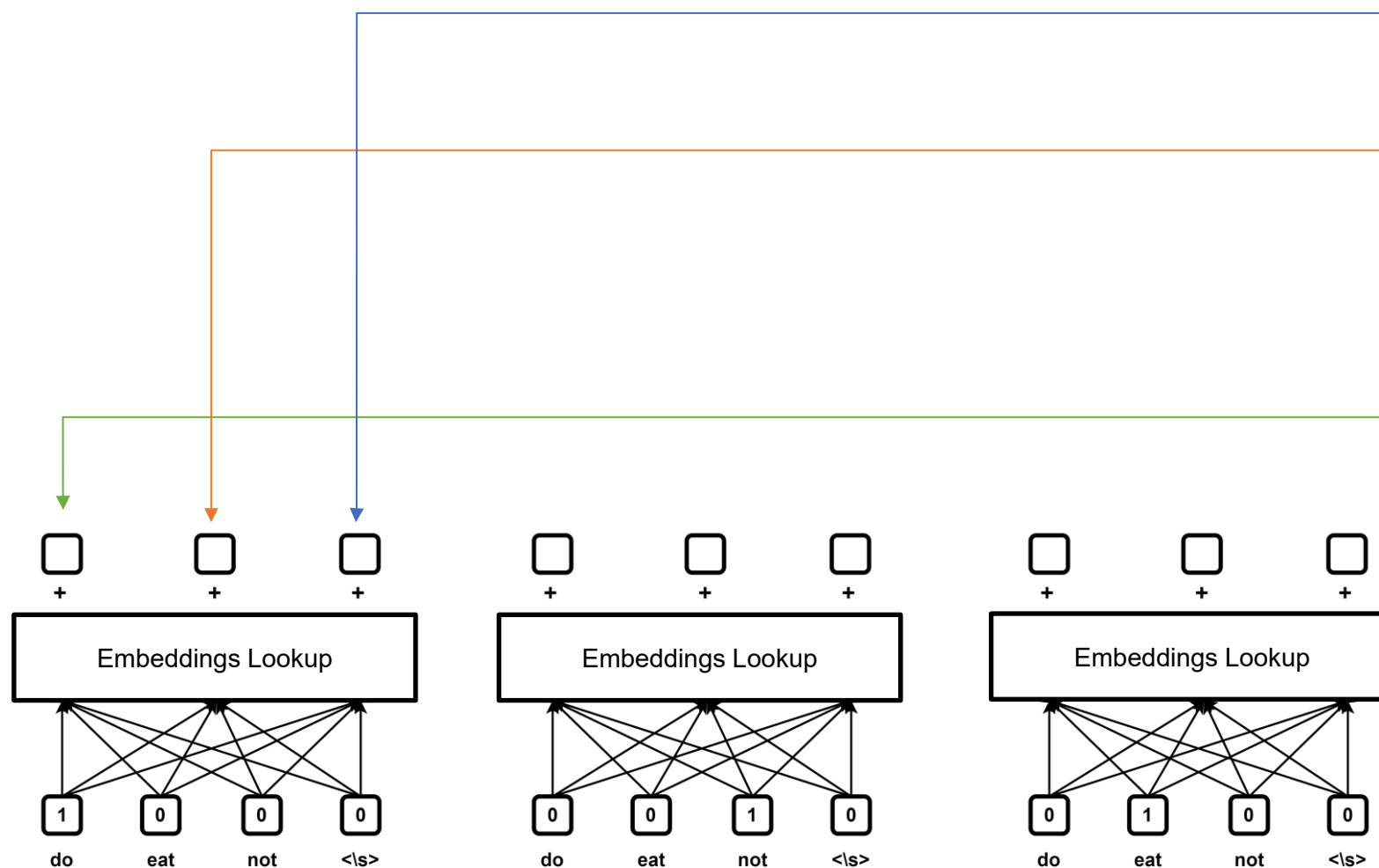
$$e^{ix} = \cos(x) + i \sin(x)$$



Growth in a **perpendicular direction** over **time** is circular: here are the **horizontal** and **vertical** coordinates.

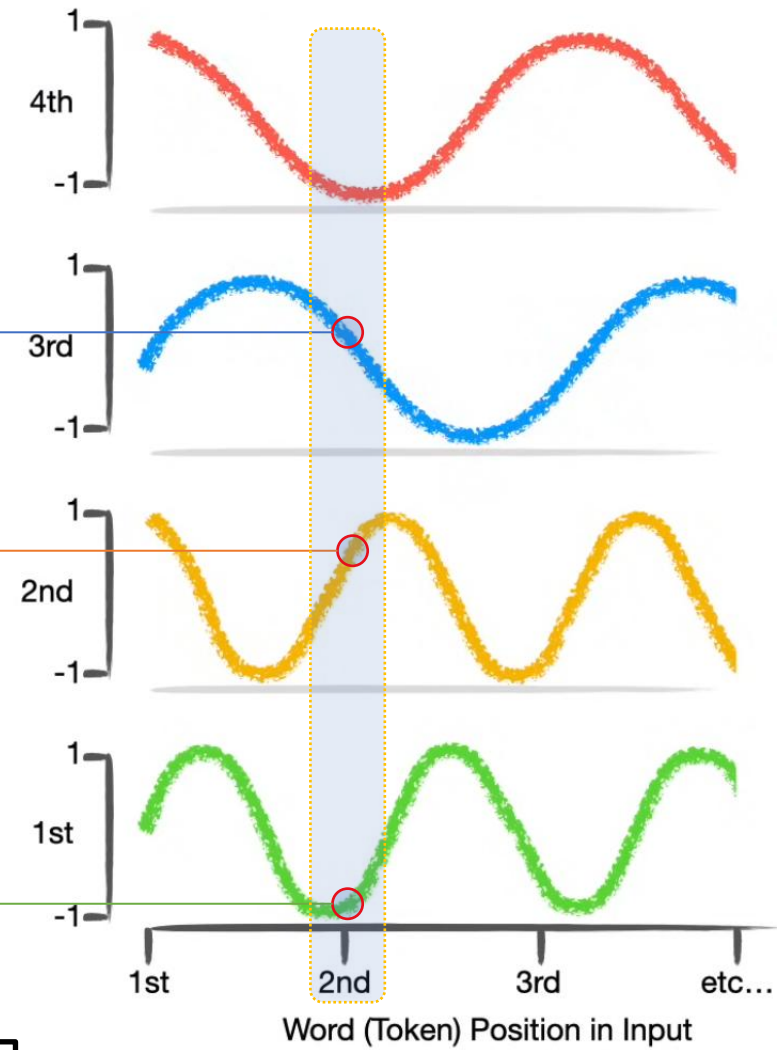
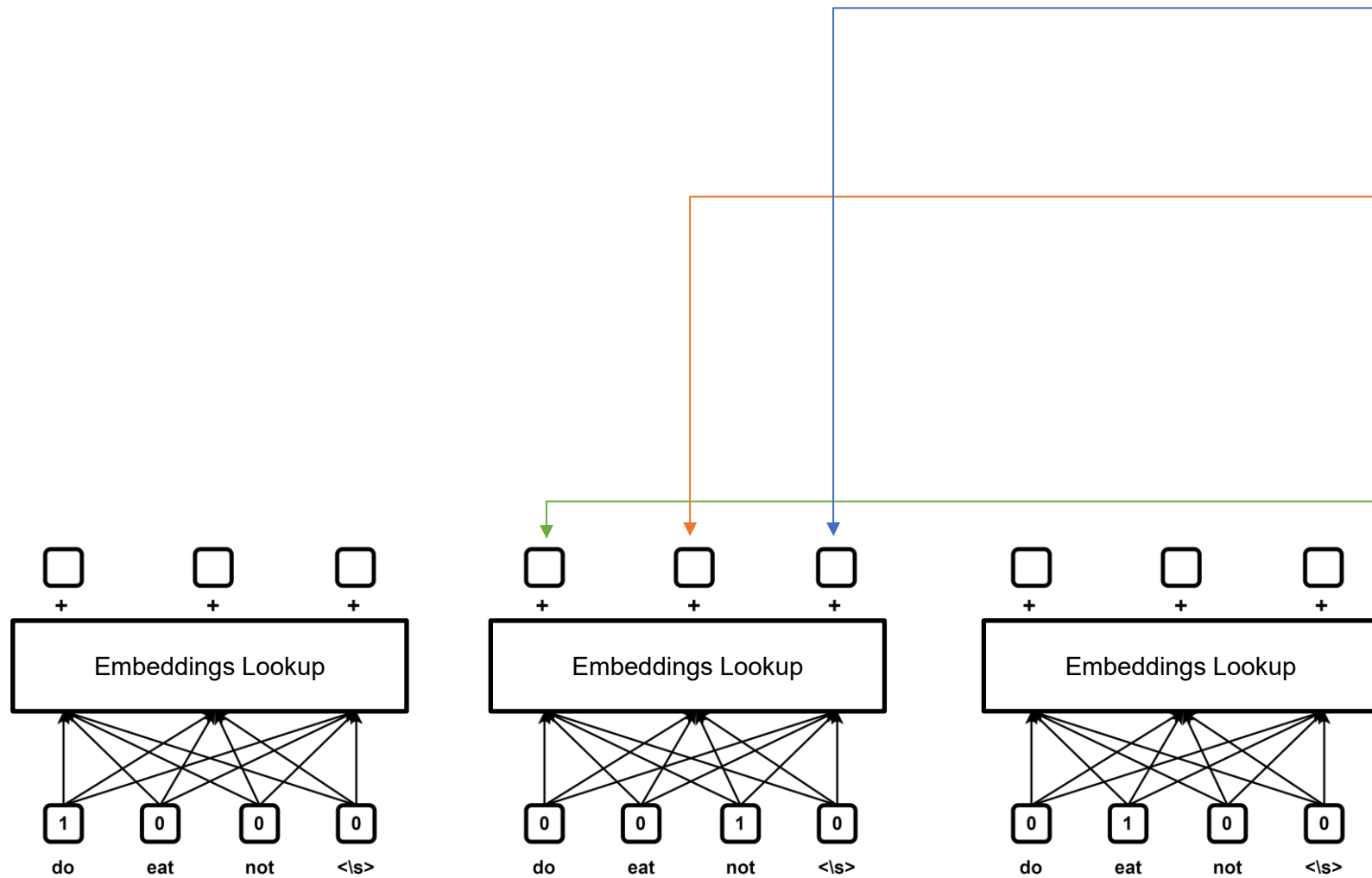
Positional Encodings

- Use alternating sines and cosines
 - With decreasing frequencies



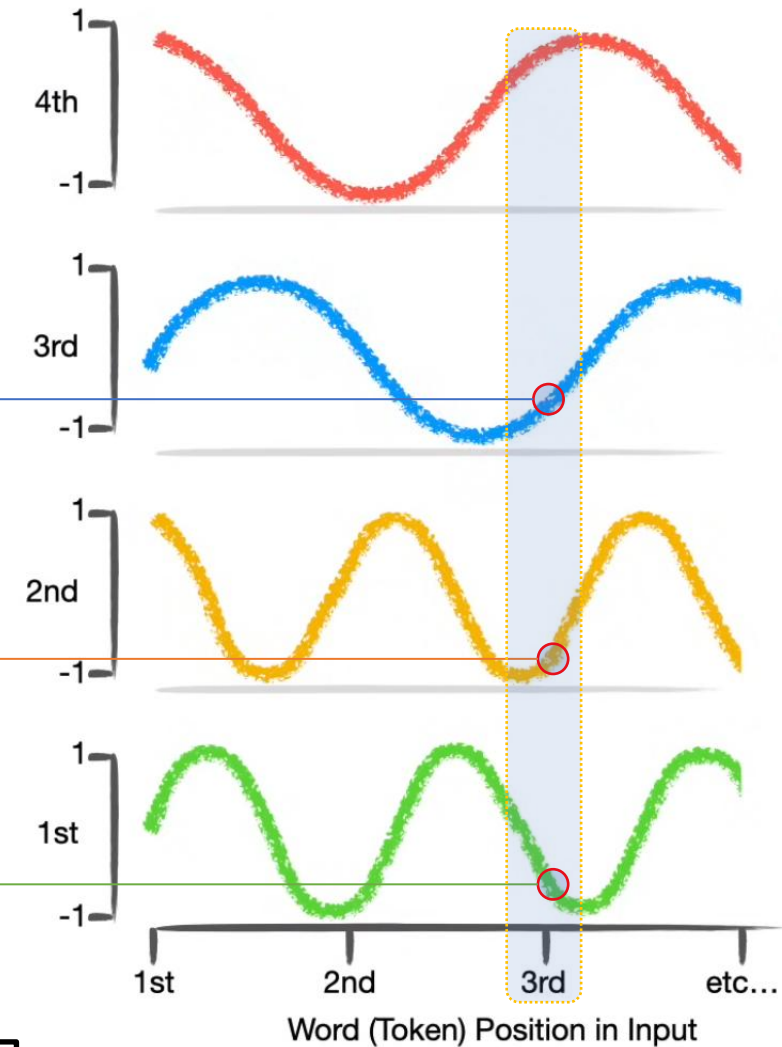
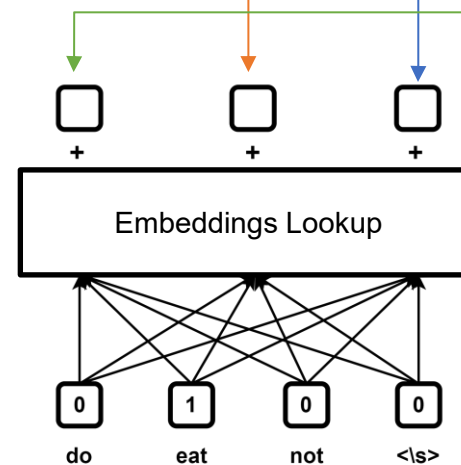
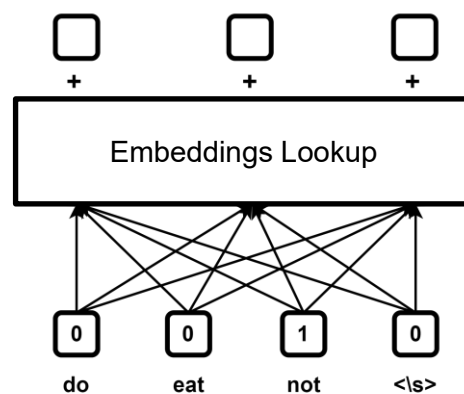
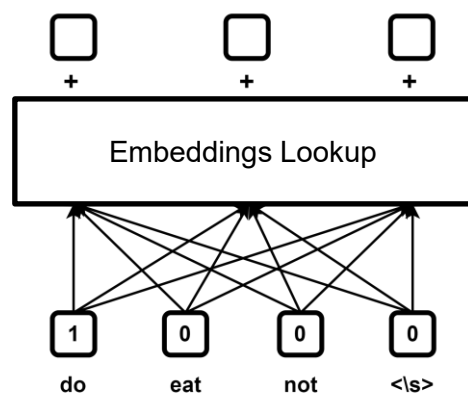
Positional Encodings

- Use alternating sines and cosines
 - With decreasing frequencies



Positional Encodings

- Use alternating sines and cosines
 - With decreasing frequencies



Sinusoidal Positional Encodings

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \quad , \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

- ***pos*** is the token's position in the sequence (0, 1, 2, ...)
- ***d_{model}*** is the dimensionality of the embedding vector (e.g., 512).
- The index ***i*** iterates over the embedding dimensions, so $2i$ and $2i + 1$ pick out even and odd dimensions respectively.
- The $2i$ and $2i + 1$ are not separate features, but representations of the same point:

Let

$$\theta_i = \frac{pos}{10000^{2i/d_{model}}}$$

From Euler's formula,

$$e^{j\theta_i} = \cos \theta_i + j \sin \theta_i$$

Therefore,

$$\boxed{PE(pos, 2i + 1) + j PE(pos, 2i) = e^{j\theta_i}}$$

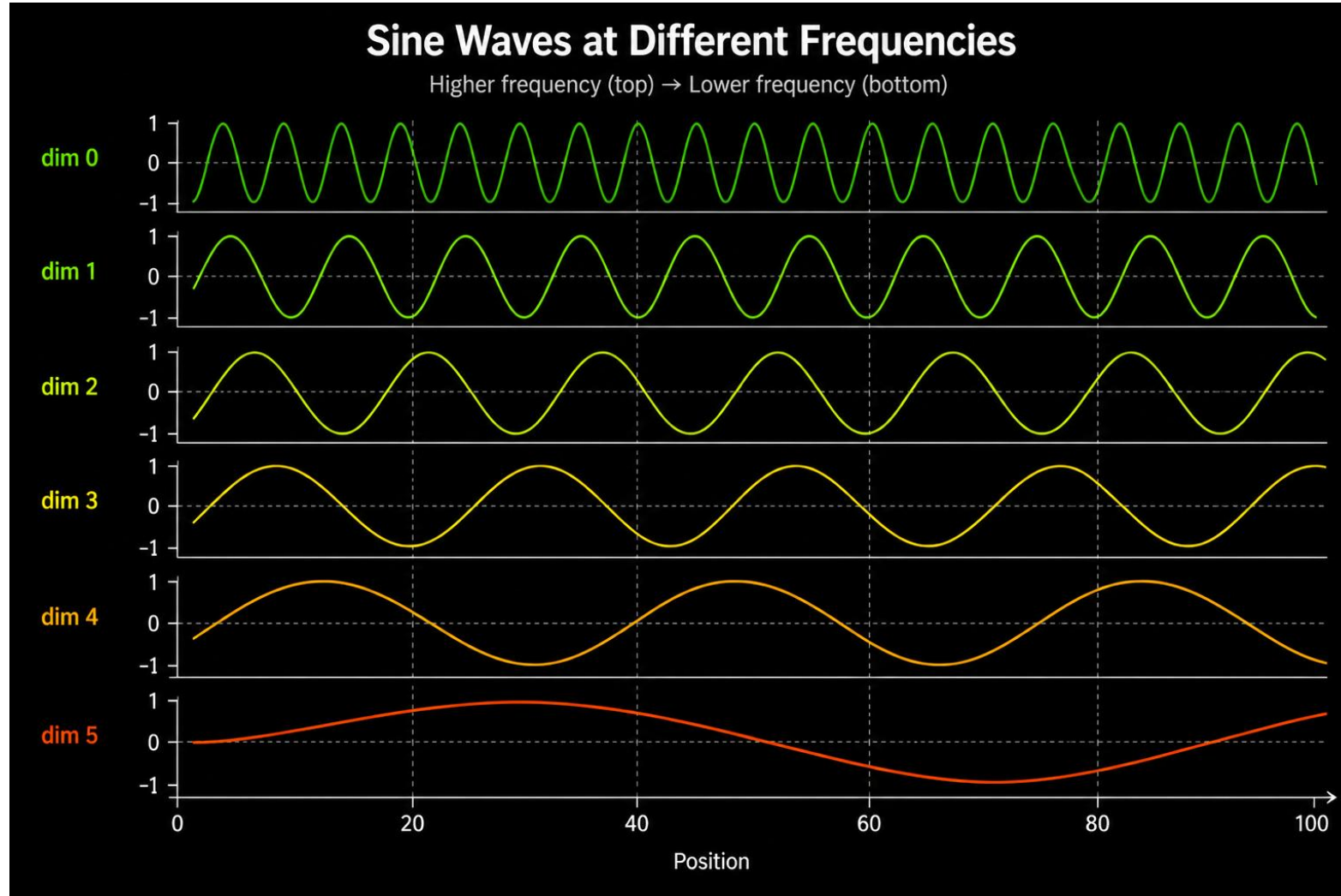
because the Transformer stores the pair as $(\sin \theta_i, \cos \theta_i)$, whereas Euler's formula gives $(\cos \theta_i, \sin \theta_i)$.

The Sinusoidal Positional Encodings

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

- Low-index dimensions oscillate rapidly (capturing fine-grained local position differences), while high-index dimensions oscillate slowly (capturing long-range position differences).
- The denominator $10000^{\frac{2i}{d_{model}}}$ controls the **wavelength/time period** of each sinusoid.
 - As i increases (moving to higher embedding dimensions), the denominator grows exponentially, which divides pos by a larger and larger number.
 - This makes the argument to sin/cos change more slowly, producing a lower-frequency wave.
 - The result is a smooth spectrum of frequencies from fast to slow across the embedding dimensions, which is what gives each position its unique fingerprint even in very long sequences.
 - The formula's real payoff, that PE_{pos+k} is a fixed linear (rotation) function of PE_{pos} for any offset k , is what makes relative position learnable by attention.

Choosing the Frequencies



This is the same multi-frequency structure we saw in binary encoding. Fast dimensions for fine-grained position. Slow dimensions for coarse position. The difference is that every wave is smooth.

Choosing the Frequencies

We need a range of frequencies. **Some should be high**, so the encoding can **distinguish nearby positions** sharply. **Some should be low**, so the encoding can carry **information across long distances without repeating**.

The paper uses this formula for the frequency of dimension pair:

$$\omega_i = \frac{1}{10000^{2i/d}}$$

Where d is the total dimensionality of the encoding.

Let us see what this gives us.

- For dimension pair $i = 0$:

$$\omega_0 = \frac{1}{10000^0} = 1$$

The wave oscillates rapidly. The value changes meaningfully with every position. This is the “fast bit” of the encoding.

- For dimension pair $i = d/2$:

$$\omega_{d/2} = \frac{1}{10000^1} = \frac{1}{10000}$$

The wave oscillates extremely slowly. It barely changes over thousands of positions. This is the “slow bit” of the encoding.

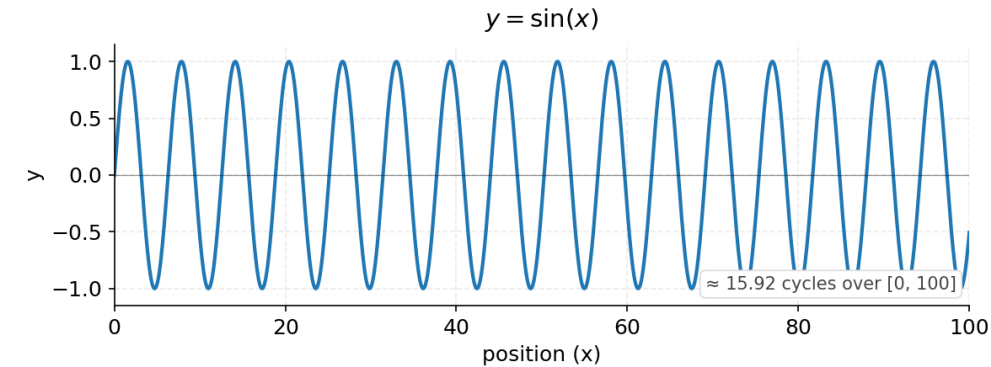
- Between these two extremes, the frequencies decrease smoothly on a geometric scale.

Choosing the Frequencies

$$y = \sin(\omega \cdot x)$$

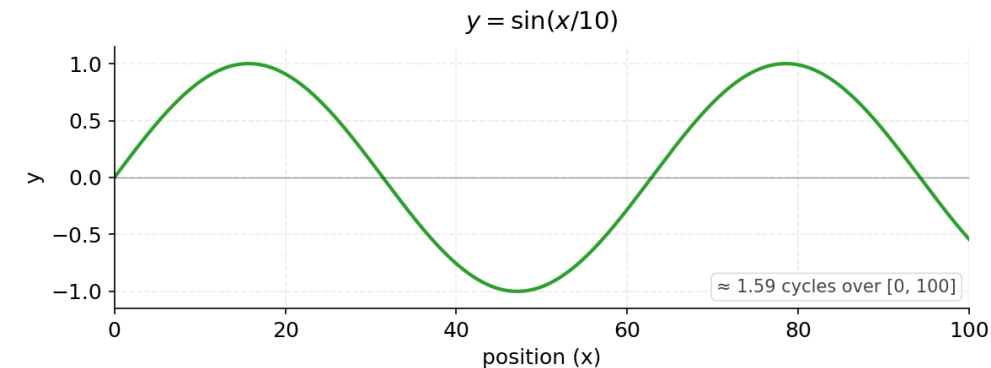
- To see this concretely, let us look at four sine waves with progressively lower frequencies.
- Consider $x = pos$ and the number multiplied to the x (pos) be the frequency ω

$$\sin(x)$$



This is the baseline. The wave completes one full cycle every 2π units. It oscillates rapidly.

$$\sin(x/10)$$



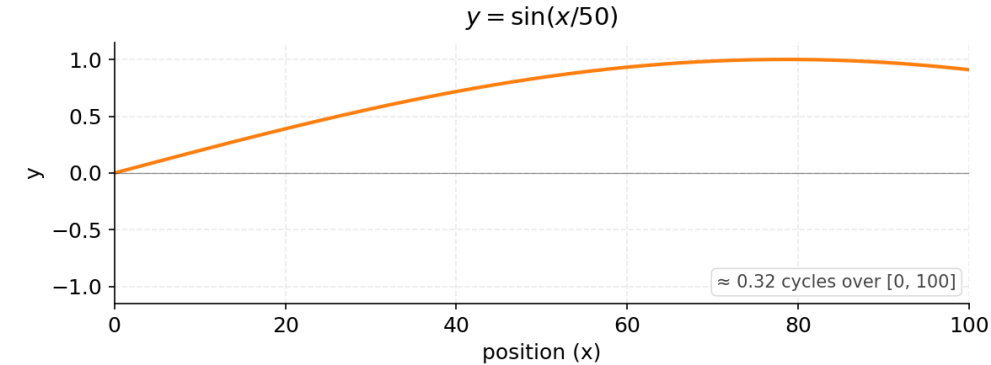
Dividing the input by 10 stretches the wave horizontally by a factor of 10. The wave still does the same thing, but it takes 10 times longer to do each cycle. One full cycle now takes about 63 i.e. $10 * 2\pi$ positions.

Choosing the Frequencies

$$y = \sin(\omega \cdot x)$$

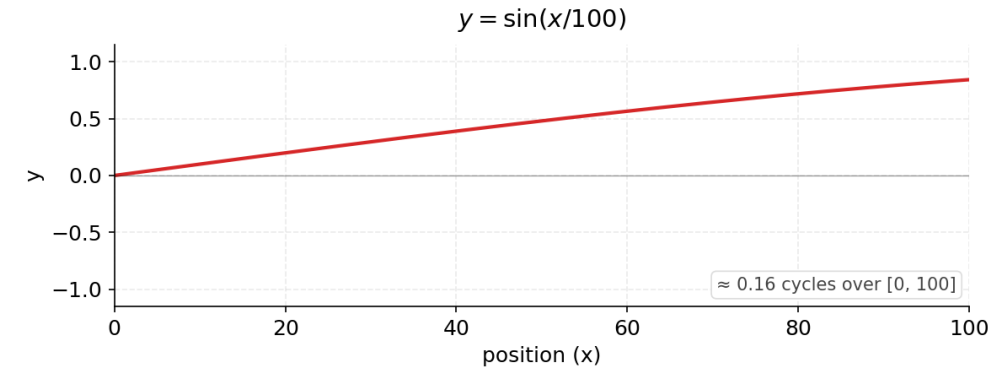
- Consider $x = pos$ and the number multiplied to the x (pos) be the frequency ω

$$\sin(x/50)$$



Now the wave is very slow. Across 100 positions, we see only about 0.32 cycles. The wave is starting to look like a gentle curve rather than a rapid oscillation..

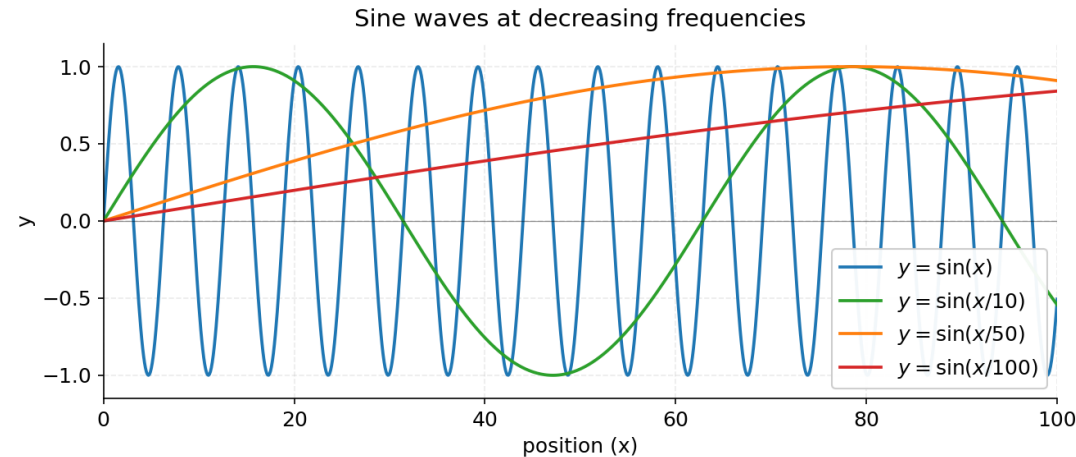
$$\sin(x/100)$$



At this frequency, we do not even complete one full cycle across 100 positions. The wave is nearly monotonic over the visible range. Adjacent positions look almost identical.

Stacking Them Together

If we stack these four waves on the same axis, we see the spectrum in one view.



- This is the same idea as binary encoding's stack of square waves, but smooth. Fast waves capture fine grained position changes. Slow waves capture coarse, sequence wide position.
- Each wave tells the model a different thing about where a token sits.
- **The actual transformer formula controls this spectrum through the choice of base 10000. Why?.**

Why Is the Base 10,000?

- From the previous slide, the slowest sinusoid has a denominator approximately equal to the base B :

$$PE_{\text{slow}}(pos) \approx \sin\left(\frac{pos}{B}\right)$$

It completes one cycle when:

$$\frac{pos}{B} = 2\pi \Rightarrow pos \approx 2\pi B$$

Base B	Slowest wave repeats after approximately
100	628 positions
10,000	62,832 positions
100 million	628 million positions

- Too small: waves repeat too soon.
- Too large: slow dimensions barely change.
- 10,000 provides a useful balance.

Why 10000?

- The number 10000 looks arbitrary. but its not.
- It controls the range of frequencies. With a base of 10000, the slowest wave completes a full cycle every $2\pi \times 10000 \approx 62,832$ positions.
 - This means within any reasonable sequence length, the slow dimensions never repeat. Every position gets a unique encoding.
- Let us plot the same encoding with three different base values: 100, 10000, and 100 million. For each base, we will look at four different dimension indices to see how the wave behaves across positions 0 to 1000.

Why 10000?

$$\sin\left(\frac{pos}{Base^{2i}}\right)$$

Each row shows one base value. Each column shows one dimension index.

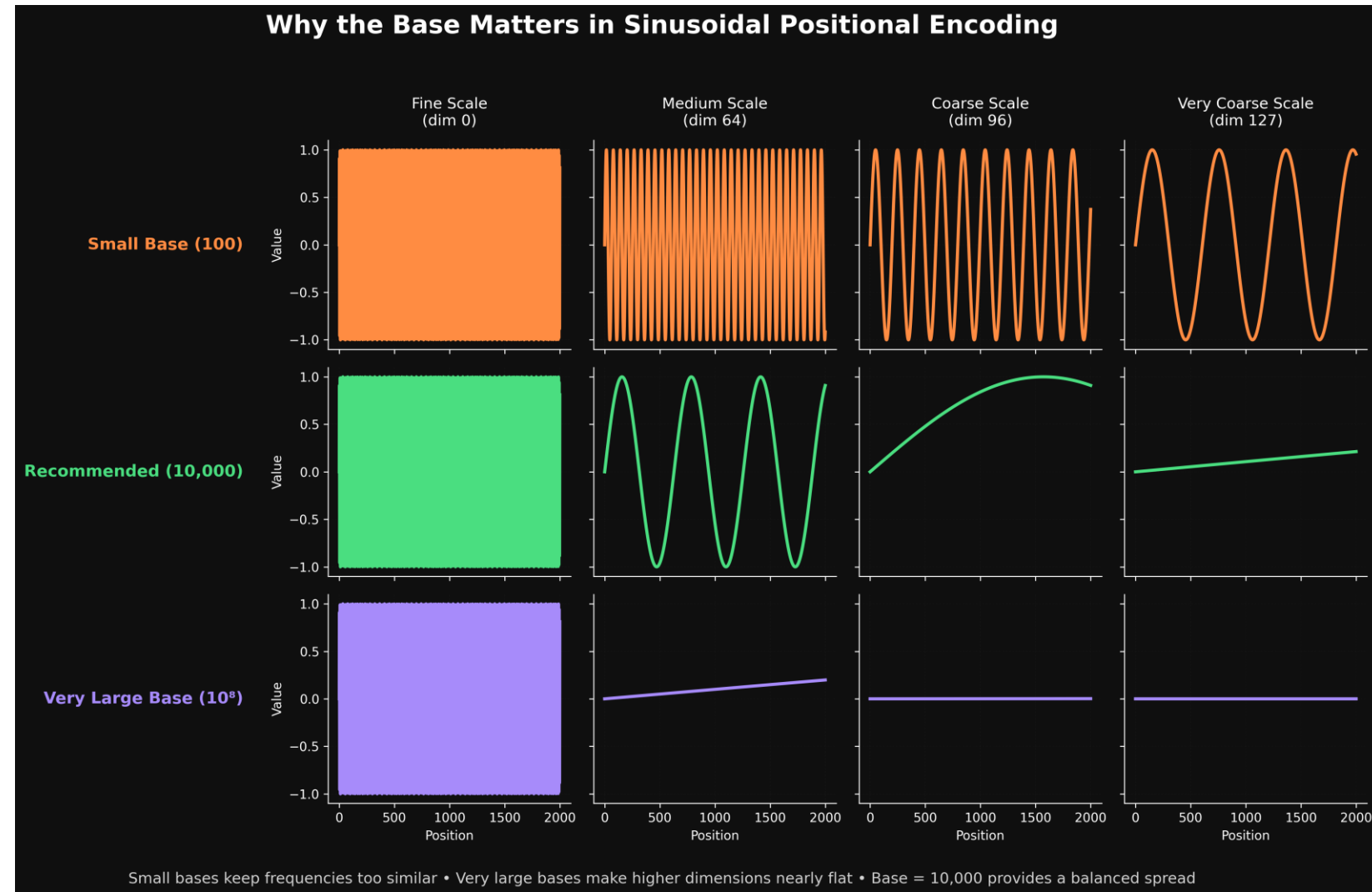
Base = 100 (top row)

- **dim 0** oscillates rapidly. This is expected. Every base produces a fast wave at dim 0.
- **dim 64** still oscillates a lot. Across 1000 positions, you see about good number of full cycles. The wave is far from slow.
- **dim 96** also oscillates clearly. About good number of cycles across 1000 positions.
- **dim 127** (the slowest dimension) still completes few full cycles in 1000 positions.

This is the problem with a small base. Even the slowest dimensions oscillate within the visible range. The waves repeat.

Two positions far apart can produce identical encodings in every dimension simultaneously.

The model loses its ability to tell distant positions apart.



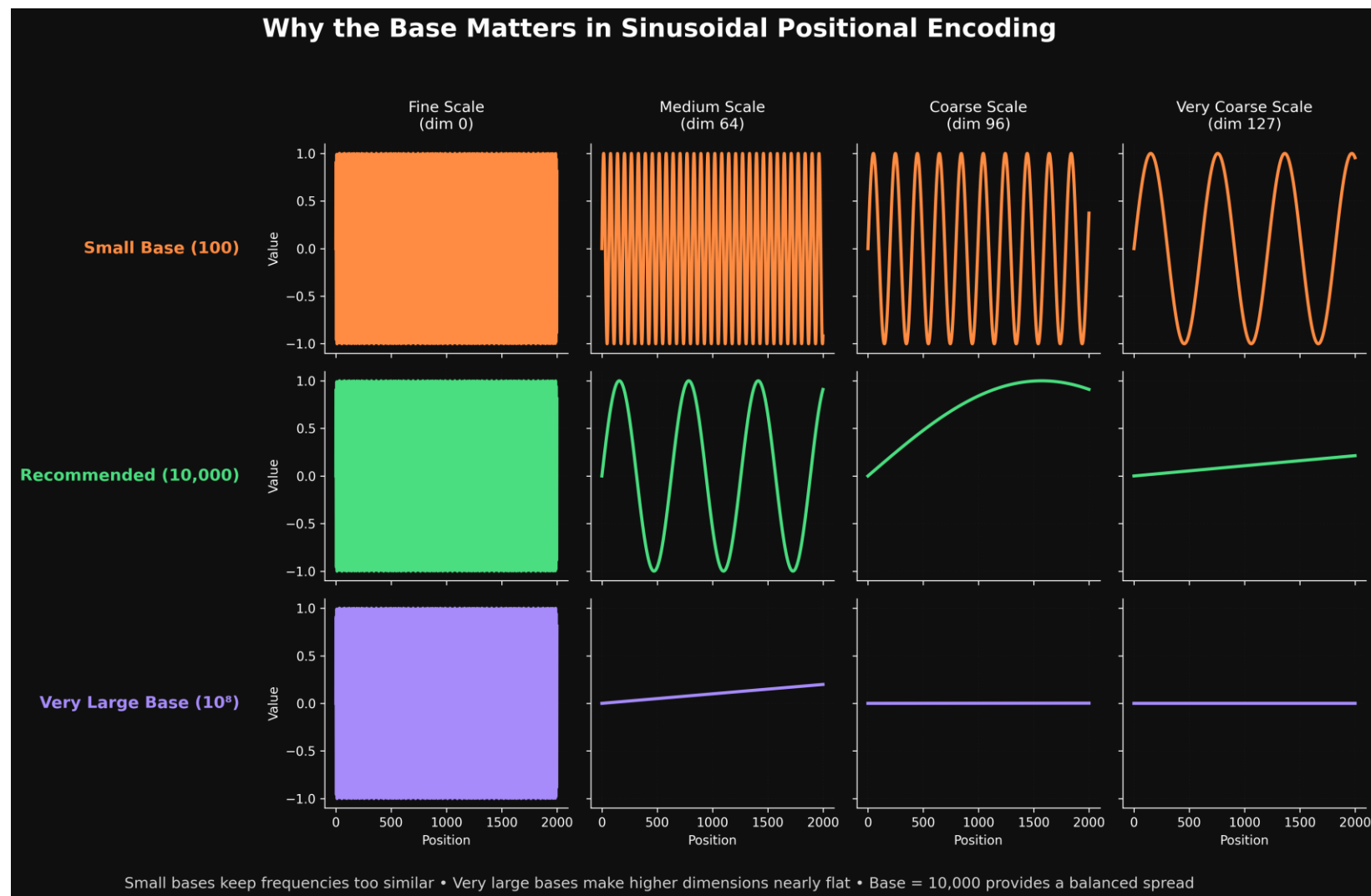
Why 10000?

$$\sin\left(\frac{pos}{Base^{\frac{2i}{256}}}\right)$$

Base = 10000 (middle row)

- This is the actual choice from the Transformer paper. Look at how the behavior changes.
- **dim 0** still oscillates rapidly. The fast end of the spectrum is unchanged.
- **dim 64** completes about 2 full cycles in 1000 positions. Slow enough to carry meaningful long-range information, fast enough to differentiate positions.
- **dim 96** does not even complete one cycle. The curve rises smoothly across the entire visible range. This dimension can distinguish between position 100 and position 800 because the values are different.
- **dim 127** is nearly flat. The value barely changes from 0 to 0.1 across 1000 positions. This is the slowest end of the spectrum.

The spread is right. Fast dimensions stay fast. Slow dimensions actually get slow. Every position in a 1000 token sequence gets a unique vector across the full encoding.



Why 10000?

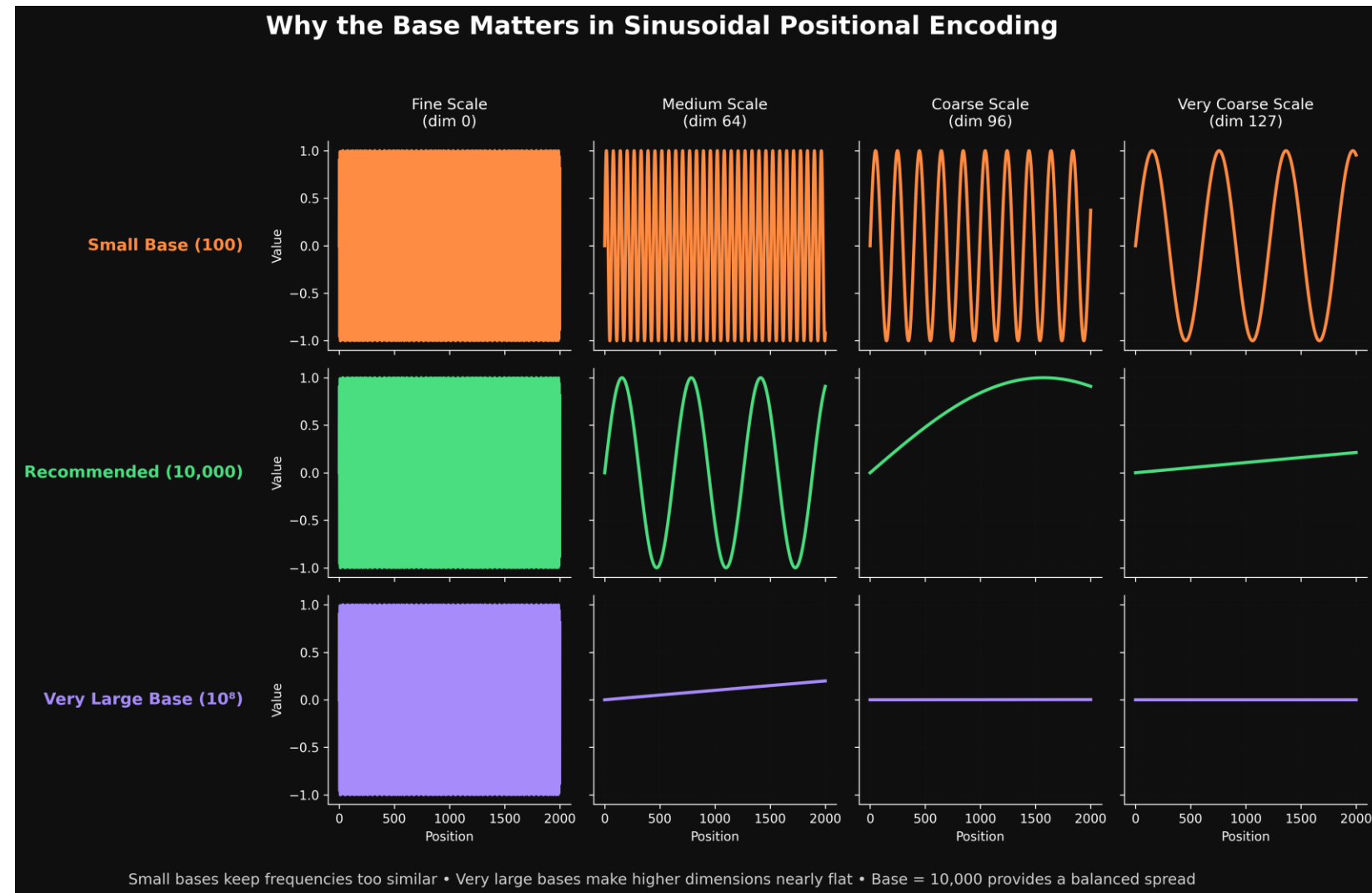
$$\sin\left(\frac{pos}{Base^{\frac{2i}{256}}}\right)$$

Base = 100 million (bottom row)

- **dim 0** still oscillates rapidly. The fast end never changes with base.
- **dim 64** is nearly a straight line. The wave has been stretched so far that it barely changes across 1000 positions.
- **dim 96** is completely flat at zero.
- **dim 127** is also completely flat at zero.

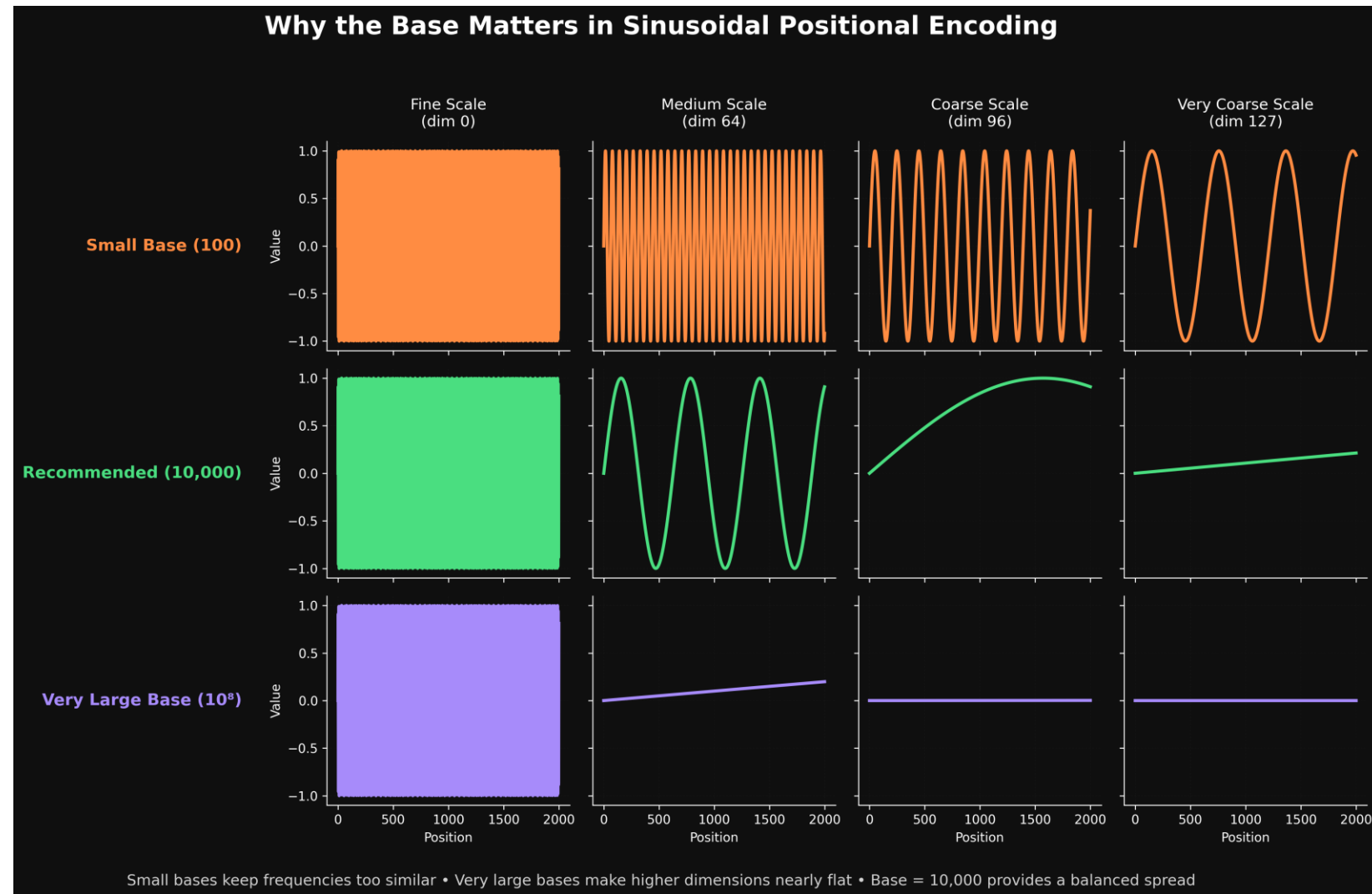
Most of the dimensions are useless. Their values are essentially the same across every position. They contribute no information about where a token is.

A base of 100 million spreads the frequency spectrum too widely. Almost the entire encoding is squeezed into the very slowest dimensions, which do nothing for typical sequence lengths.

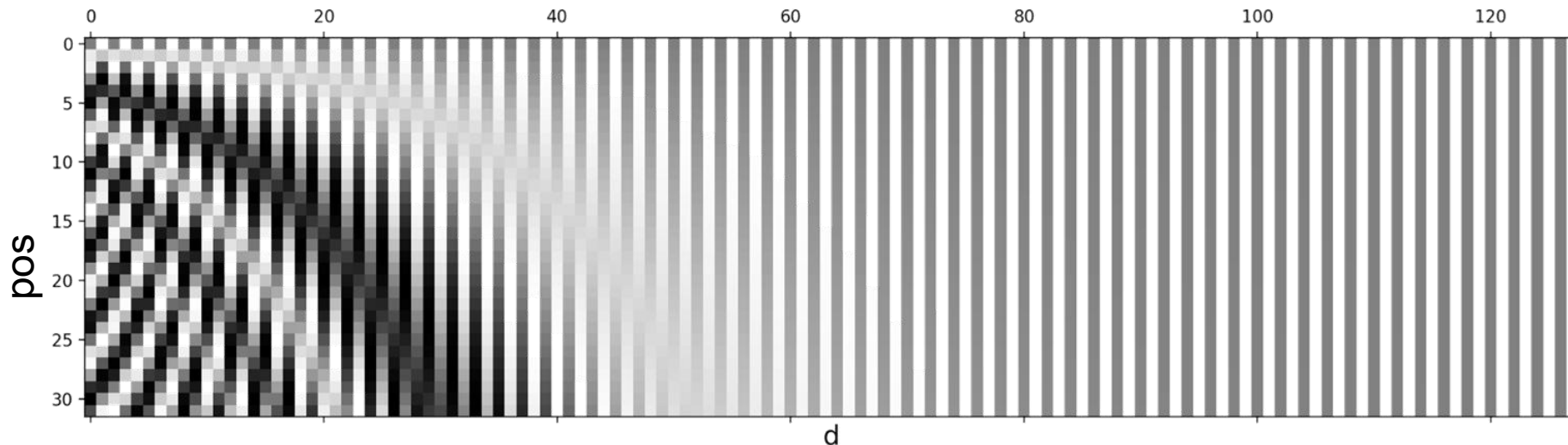


Why 10000?

- A smaller base, like 100, would cause the slow waves to repeat much sooner.
 - Different positions would start getting identical encodings.
 - The model would lose the ability to tell them apart.
- A larger base, like a million, would spread the frequencies too thin.
 - Most dimensions would oscillate too slowly to be useful.
- The value 10000 was chosen to balance these two concerns.
 - The Goldilocks's Choice.
 - It is large enough to avoid repetition within typical context lengths, but not so large that the frequency spectrum becomes useless.



Visualizing Positional Encodings



- This is the positional encoding matrix visualized as a heatmap
 - Each row is a token position (pos, y-axis)
 - Each column is an embedding dimension (d, x-axis)
 - With brightness representing the sin/cos value.
- On the far left (low dimensions), you see a rapid checkerboard pattern.
 - These are the high-frequency sinusoids that oscillate fast across positions, letting the model distinguish adjacent tokens.
- As you move rightward, the stripes get progressively wider and more vertical.
 - The frequencies slow down, so many neighboring positions share similar values, capturing long-range structure.
- By the far right (dimensions ~80+), the pattern looks like near-uniform vertical bars because those sinusoids are so slow that all 32 positions shown here barely sample a tiny fraction of one cycle.
- **The key insight:** Every row (position) has a unique horizontal fingerprint across all dimensions, even though no single dimension alone would be sufficient. It's the combination of fast, medium, and slow waves that creates a unique encoding for each position.

Why Both Sine and Cosine?

- We now have a smooth multi frequency encoding. Every position gets a unique vector. The values are bounded between -1 and 1. The transitions between adjacent positions are continuous.
- This is already a complete positional encoding. We could stop here **and use just sine waves**.
- But the original Transformer paper does not stop here. Half of the dimensions use sine. The other half use cosine.
- Why? What does cosine give us that sine alone does not?
 - We have a working positional encoding using only sine waves. Each position gets a vector. The values are bounded and smooth. The frequencies span from fast to slow.
 - So why does the original Transformer paper use sine for half the dimensions and cosine for the other half?
 - The formula assigns sine to even indexed dimensions and cosine to odd indexed dimensions:
$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Cosine looks redundant. It is just a shifted sine wave. So why include it?

- The reason is a property called **linear transformability**.

Linear Transformability

- We want the encoding to have a useful structure.
 - Given the encoding of one position, we want to reach the encoding of any other position using a simple linear operation.
 - If we know the encoding at position pos , we want to compute the encoding at position $pos + k$ by just multiplying with a matrix.
 - And the same shift should always use the same matrix.
 - This is a strong property. If it holds, the model can reason about relative shifts using simple linear layers.
 - This matters because W_q and W_k are exactly that, linear layers.

The Pair Representation

- Take a single frequency θ . Represent each position as a pair of values, one sine and one cosine:
$$PE(pos) = (\sin(pos \theta), \cos(pos \theta))$$
- Now, let us ask one question:
 - Can we get from $PE(pos)$ to $PE(pos + k)$ using a fixed matrix?

Linear Transformability

Write down the encoding at position $pos + k$:

$$PE(pos + k) = (\sin((pos + k)\theta), \cos((pos + k)\theta))$$

Expand each term using the angle addition formulas:

$$\sin((pos + k)\theta) = \sin(pos\theta) \cos(k\theta) + \cos(pos\theta) \sin(k\theta)$$

$$\cos((pos + k)\theta) = \cos(pos\theta) \cos(k\theta) - \sin(pos\theta) \sin(k\theta)$$

Two rules were used:

- **Distributive property**

$$(pos + k)\theta = pos\theta + k\theta$$

- **Angle-addition identities**

$$\sin(A + B) = \sin A \cos B + \cos A \sin B$$

$$\cos(A + B) = \cos A \cos B - \sin A \sin B$$

- with $A = pos\theta$ and $B = k\theta$.

Look at the right-hand side. Both new values are built only from $\sin(pos\theta)$ and $\cos(pos\theta)$. They are scaled by $\cos(k\theta)$ and $\sin(k\theta)$. This is exactly a matrix multiplication:

$$\begin{bmatrix} \sin((pos + k)\theta) \\ \cos((pos + k)\theta) \end{bmatrix} = \begin{bmatrix} \cos(k\theta) & \sin(k\theta) \\ -\sin(k\theta) & \cos(k\theta) \end{bmatrix} \begin{bmatrix} \sin(pos\theta) \\ \cos(pos\theta) \end{bmatrix}$$

Call this matrix :

$$\mathbf{M}_k = \begin{bmatrix} \cos(k\theta) & \sin(k\theta) \\ -\sin(k\theta) & \cos(k\theta) \end{bmatrix}$$

So we have:

$$PE(pos + k) = \mathbf{M}_k \cdot PE(pos)$$

The matrix $\mathbf{M}_k$ depends only on the shift k . It does not depend on pos . The same shift always uses the same matrix.

This is the property we wanted.

Why Sine Alone Cannot Do This

Now try the same thing with only sine.

- Suppose the encoding is just $\sin(pos \theta)$, a single value.
- To get the encoding at $pos + k$, we need:

$$\sin((pos + k)\theta) = \sin(pos \theta) \cos(k\theta) + \cos(pos \theta) \sin(k\theta)$$

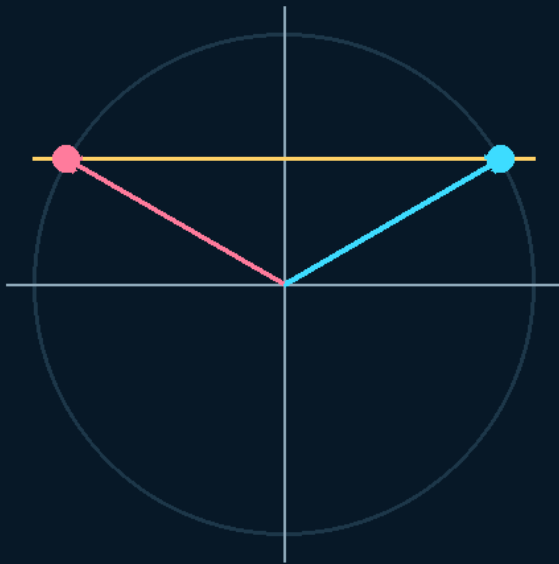
Look at the right-hand side. It needs $\cos(pos\theta)$.

- But a sine only encoding does not store $\cos(pos\theta)$. We only have $\sin(pos\theta)$. The term we need is missing.
- There is no way to recover $\cos(pos\theta)$ from $\sin(pos\theta)$ using a linear operation.
 - So the shift cannot be written as a fixed matrix.
 - **The transformation is impossible with sine alone.**

Cosine from Sine?

Because one sine value does not uniquely determine phase.

Why sine alone is not enough



Both points have $\sin \theta = 0.50$
but $\cos \theta$ is $+0.87$ or -0.87

same y-value $\rightarrow$ two phases



SAME SINE, DIFFERENT COSINE

$$\sin^2 \theta + \cos^2 \theta = 1$$

Therefore,

$$\cos \theta = \pm \sqrt{1 - \sin^2 \theta}$$

INTERPRETATION

A single projection loses which side of the circle the point is on.
Two coordinates recover the phase.

How Sinusoidal Encoding Enters the Model

So far, we have studied the positional encoding on its own.

- But there is a question we have not asked yet.
 - How does this encoding actually get used inside the Transformer?
- And the answer reveals a hidden problem!

Properties

- **Dot product property:**

- The dot product of two unit vector encodings gives $\cos(\theta(m - n))$, which depends only on relative position. (As, $A \cdot B = |A||B|\cos(\theta)$, where θ is the angle between the two vectors)
- This property matters inside the attention mechanism, where queries and keys are multiplied together.
- See next slide

- **Linear shift (transformability) property:**

- A fixed matrix M_k can shift an encoding from one position to another. This property matters for the linear layers in the model, such as W_q and W_k .

These are two separate capabilities. They both need sine and cosine, but for different mathematical reasons.

$$\mathbf{p}_m^\top \mathbf{p}_n = \cos(\theta(m - n))$$

For one sine–cosine pair,

$$\mathbf{p}_m = \begin{bmatrix} \sin(m\theta) \\ \cos(m\theta) \end{bmatrix}, \mathbf{p}_n = \begin{bmatrix} \sin(n\theta) \\ \cos(n\theta) \end{bmatrix}.$$

Their dot product is:

$$\begin{aligned} \mathbf{p}_m^\top \mathbf{p}_n &= [\sin(m\theta) \ \cos(m\theta)] \begin{bmatrix} \sin(n\theta) \\ \cos(n\theta) \end{bmatrix} \\ \mathbf{p}_m^\top \mathbf{p}_n &= \sin(m\theta)\sin(n\theta) + \cos(m\theta)\cos(n\theta) \end{aligned}$$

Using

$$\begin{aligned} \cos(A - B) &= \cos A \cos B + \sin A \sin B \\ \mathbf{p}_m^\top \mathbf{p}_n &= \cos(\mathbf{m}\theta - \mathbf{n}\theta) = \cos(\theta(\mathbf{m} - \mathbf{n})) \end{aligned}$$

Because both vectors have norm 1 ($\|\mathbf{p}_m\| = \|\mathbf{p}_n\| = 1$), this dot product is also their **cosine similarity**.

For the complete positional encoding, the dot product is a sum across frequencies:

$$PE_m^\top PE_n = \sum_i \cos(\theta_i(m - n))$$

Position Is **Added** to the Embedding

- In the original Transformer, the positional encoding is added directly to the token embedding before anything else happens.
- For a token at position m :

$$\text{input}_m = \text{embed}_m + PE_m$$

- The embedding carries the meaning of the token. The encoding carries its position. We add them together into a single vector.
- This combined vector is what flows into the attention mechanism. The model then computes queries and keys from it:

$$Q_m = W_q \cdot (\text{embed}_m + PE_m) = W_q \cdot \text{embed}_m + W_q \cdot PE_m$$

$$K_n = W_k \cdot (\text{embed}_n + PE_n) = W_k \cdot \text{embed}_n + W_k \cdot PE_n$$

Each query and key now has two parts. A semantic part from the embedding, and a positional part from the encoding.

And the attention mechanisms will determine the attention scores based on these Q_m and K_n vectors.

Attention Score Expansion

Now we compute the attention score. The score is the dot product of Q_m and K_n .

- Both Q_m and K_n have two parts. When we multiply two sums, every part of the first multiplies every part of the second.

$$Q_m \cdot K_n = (W_q \cdot \text{embed}_m + W_q \cdot PE_m) \cdot (W_k \cdot \text{embed}_n + W_k \cdot PE_n)$$

Expanding gives four terms:

$$\begin{aligned} Q_m \cdot K_n &= (W_q \cdot \text{embed}_m) \cdot (W_k \cdot \text{embed}_n) && \text{Term 1} \\ &+ (W_q \cdot \text{embed}_m) \cdot (W_k \cdot PE_n) && \text{Term 2} \\ &+ (W_q \cdot PE_m) \cdot (W_k \cdot \text{embed}_n) && \text{Term 3} \\ &+ (W_q \cdot PE_m) \cdot (W_k \cdot PE_n) && \text{Term 4} \end{aligned}$$

Let us read each term:

Term 1 is **purely semantic**. It is the meaning of token m against the meaning of token n . No position involved.

Term 2 is a cross term. The meaning of token m against the position of token n . Semantic mixed with position.

Term 3 is the other cross term. The position of token m against the meaning of token n . Position mixed with semantic.

Term 4 is **purely positional**. The position of token m against the position of token n . This is the term that contains $\cos(\theta(m - n))$, **the clean relative position signal**.

The relative position information we worked so hard to derive lives **only in Term 4**. And **even term 4 is not guaranteed to remain purely relative** after arbitrary W_q, W_k projections. The clean relative-distance identity holds for the unprojected sinusoidal dot product; those projections can disrupt it.

The Entanglement Problem

The model **never sees *Term 4* by itself**. It sees the sum of all four terms. **The clean relative position signal is buried inside a mixture.**

- Terms 2 and 3 are the troublemakers.
 - They mix semantic content with positional content.
 - They are noise sitting on top of the signal the model actually wants.
- The model has to work through this mixture.
 - There is no part of the architecture that isolates Term 4.
 - **The model must learn, on its own, how to make use of the relative position signal while ignoring the cross terms.**
- This creates a heavy burden on W_q and W_k .
 - These two matrices must do two jobs at once.
 - They must project semantic meaning into a useful space.
 - And they must preserve the positional structure, so the relative position signal survives.
 - Two competing goals, packed into one set of weights.

Absolute Versus Relative Position

There is a deeper issue here:

- The encoding PE_m stores absolute position.
 - PE_5 is a fixed vector. It means position 5 and nothing else, no matter what sequence it appears in.
- But what the model actually wants is relative position.
 - It wants to know that two tokens are 9 apart, not that one is at position 5 and the other at position 14.
- Relative position is not stored anywhere.
 - It only appears as a byproduct, inside Term 4, after the dot product is computed.
 - It is never represented directly.
- **So the model is given absolute positions and asked to figure out relative positions on its own.**
 - It can do this, but only by learning. There is no guarantee it learns it perfectly.

Position Is Fused With Meaning

There is one more limitation, and it is structural.

- Once we compute $\text{embed}_m + PE_m$, the two parts are added into a single vector.
 - They cannot be pulled apart again.
 - Addition removes the boundary between them.
- **Every layer** after this point sees one fused vector.
 - It cannot choose to look at only the meaning, or only the position.
 - The two are tangled together for the rest of the network.
- Sometimes the later stages of the transformer only need meaning. Sometimes they only need position. But they cannot separate them. They are stuck with the mixture.

Summary: Limitations of Sinusoidal Encodings

Sinusoidal encoding is **smooth, bounded, unique, and carries relative position inside its dot product**. It was a real step forward.

But it has three weaknesses:

- **Unstructured interaction:** Adding position creates cross terms involving content and absolute position.
- **No guaranteed relative-only score:** Sinusoidal encodings can express relative relationships, but learned Q/K projections need not preserve them.
- **Additive fusion:** Content and position enter the projections as a sum; there is no explicit separation. And the later layers are forced to handle the mixture.

All three problems share one root cause: **Position is added to the embedding.**

- What if we did not add position at all?
- What if, instead of adding a positional vector, we applied position as an operation **directly on the query and key before attention?**

This is the idea behind RoPE.

Rotary Position Embeddings (RoPE)

RoPE, introduced by *Su et al.* in 2021, is the positional encoding used in almost every modern large language model, including **LLaMA, Mistral, Gemma, and Phi**.

The Design Goal

- In sinusoidal encoding, the attention score expanded into four terms.
 - Only one of them carried clean relative position.
 - The other two were noise that mixed meaning with position.
- We want something better.
 - We want the attention score to look like this:
$$Q_m \cdot K_n = f(\text{semantics}, m - n)$$
 - One clean expression.
 - The score should depend on the meaning of the two tokens and on their relative distance $m - n$.
 - Nothing else. No cross terms. No entanglement.

RoPE: The Key Insight

The problem with sinusoidal encoding was the **order of operations**.

- We added position to the embedding **first**.
 - **Then** we multiplied by W_q and W_k .
 - Because position was already mixed into the embedding, the multiplication produced cross terms.
- RoPE flips the order.
 - It does not touch the embedding. Instead, it lets W_q and W_k do their work first, producing the query and key.
 - Then it applies position directly to those output vectors.
 - **This happens before attention scores are calculated**, which can cater to the relative positions of the tokens
- In other words: apply position **after** the projection, **not before**.

RoPE: The Benefits

When position is applied after W_q and W_k , those two matrices no longer have to deal with position at all.

- Their only job is to handle meaning.
- They project the token embedding into a query or key that captures semantic content.
- Position becomes a separate, independent step.
 - It is applied on top of the query and key as its own operation.
- There are no longer two competing goals packed into W_q and W_k .
 - The projection handles meaning.
 - The positional operation handles position.
 - Each does one job.

Now the question is: **What operation should we apply to the query and key to inject position?**

- RoPE's answer is **rotation**.

The Rotation Operation

Take a query vector. To keep things simple, let's start with just two dimensions.

$$Q = (q_0, q_1)$$

This query sits at position m . We pick a frequency θ , the same kind of frequency we used in sinusoidal encoding.

To inject position, we rotate this 2D vector by an angle of $m\theta$. The rotation is done with a rotation matrix:

$$\begin{bmatrix} q_0^{new} \\ q_1^{new} \end{bmatrix} = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \end{bmatrix}$$

- This uses the same sine and cosine values as sinusoidal encoding.
 - But the operation is different.
 - We are not adding anything.
 - We are rotating the vector.
- **The position m decides how much we rotate.**
 - A token at position 1 is rotated by θ .
 - A token at position 2 is rotated by 2θ .
 - A token at position 100 is rotated by 100θ .
 - The further along the sequence, the more the vector turns.

The Rotation Operation

Let us expand the matrix multiplication to see the new values directly:

$$q_0^{new} = q_0 \cos(m\theta) - q_1 \sin(m\theta)$$

$$q_1^{new} = q_0 \sin(m\theta) + q_1 \cos(m\theta)$$

The new query is a mix of the old components, weighted by sine and cosine of the rotation angle.

The length of the vector does not change. Rotation only turns the vector; it does not stretch or shrink it.

The meaning carried by the magnitude stays intact. Only the direction shifts, and the amount of shift encodes the position.

We do the exact same thing to the key vector, using its position n :

$$k_0^{new} = k_0 \cos(n\theta) - k_1 \sin(n\theta)$$

$$k_1^{new} = k_0 \sin(n\theta) + k_1 \cos(n\theta)$$

Now both the query and the key have been rotated by their own positions.

The next question is what happens when we take the dot product of two rotated vectors.

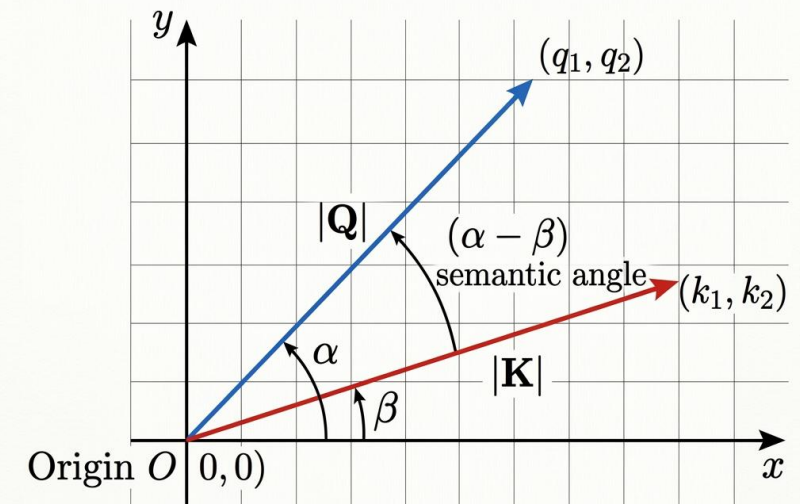
Rotation Gives Pure Relative Position

We have rotated the query by its position and the key by its position. Now we take their dot product and see what comes out. This is the heart of RoPE.

- Every 2D vector has a direction, which we can describe with an angle.
- Let the query Q point in direction α . This angle captures the semantic content of the query, the meaning that W_q produced.
- Let the key K point in direction β . This angle captures the semantic content of the key.
- Before any rotation, the dot product of two unit vectors depends on the angle between them:

$$Q \cdot K = \cos(\alpha - \beta)$$

The score depends on $\alpha - \beta$, the angle between the two directions. This is the semantic relationship between the query and the key.



the dot product depends only on the angle between Q and K .

The Rotation Operation

Now we rotate. The query is at position m , so we turn it by $m\theta$. The key is at position n , so we turn it by $n\theta$.

Rotation simply adds to the angle. After rotation:

Q_m points in direction $\alpha + m\theta$

K_n points in direction $\beta + n\theta$

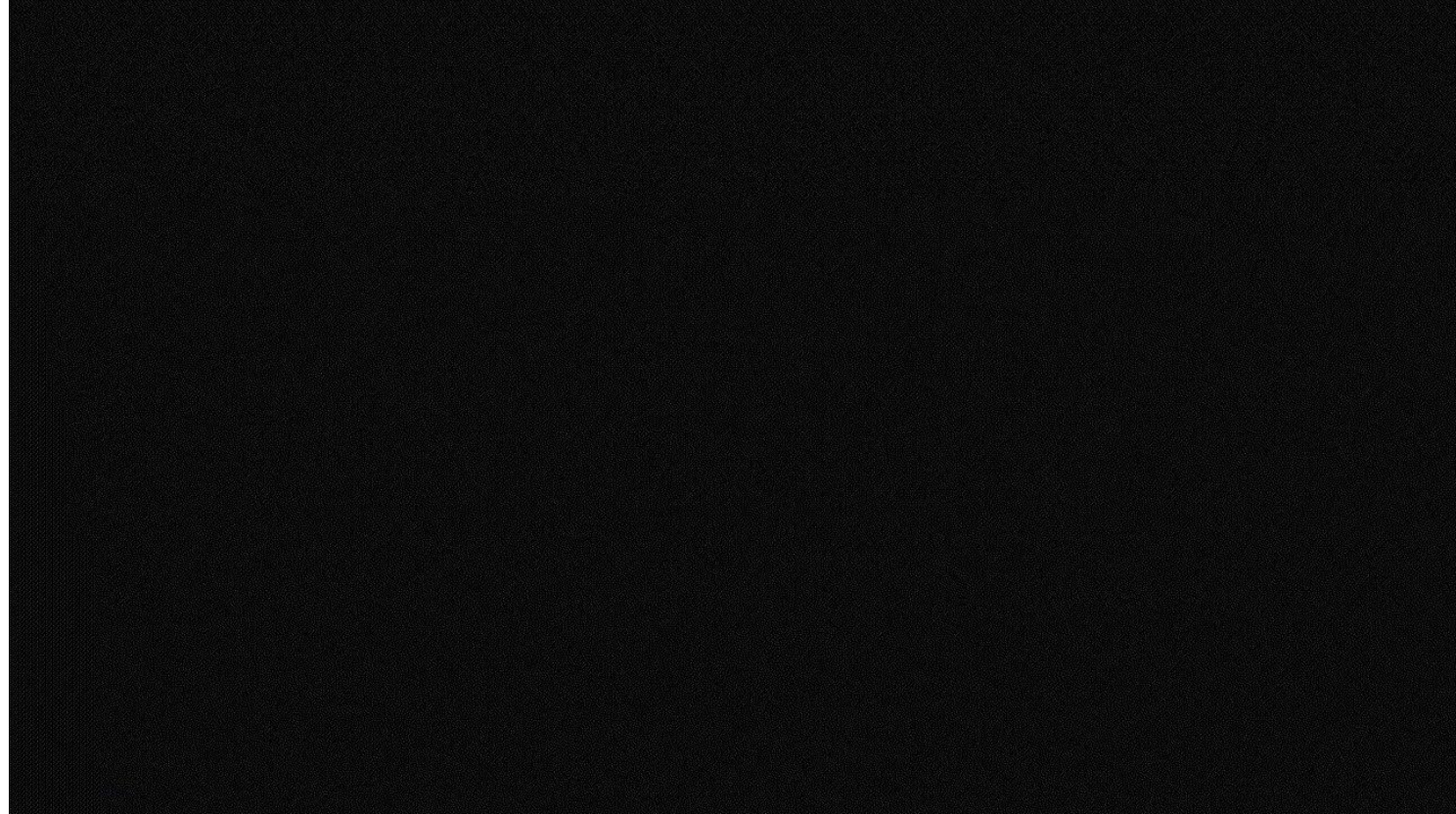
Take the dot product of the rotated vectors. It depends on the angle between them, just like before:

$$Q_m \cdot K_n = \cos((\alpha + m\theta) - (\beta + n\theta))$$

Simplify the inside:

$$Q_m \cdot K_n = \cos((\alpha - \beta) + (m - n)\theta)$$

Look at this result carefully.



[Positional Encoding Explained: Sinusoidal Embeddings and RoPE \(Part 2\) | Naveen Reddy](#)

The Rotation Operation

$$Q_m \cdot K_n = \cos((\alpha - \beta) + (m - n)\theta)$$

Inside the cosine there are two pieces:

- $(\alpha - \beta)$ is the semantic relationship.
 - It is exactly the same angle that was there before rotation. It is untouched.
- $(m - n)\theta$ is the positional piece.
 - It depends only on $m - n$, the relative distance between the two tokens.
- The semantic part and the positional part sit side by side inside a single expression.
 - The meaning is preserved.
 - The position is relative.
 - There is nothing to disentangle.
- This is exactly the goal we set at the start.
 - The attention score depends only on the semantics and on the relative distance .

Notes: The Same Result Through Matrices

The angle argument is good, but let us confirm it with the matrices directly.

Write the rotated query as $R_m Q$ and the rotated key as $R_n K$, where R_m and R_n are rotation matrices. Their dot product is:

$$Q_m \cdot K_n = (R_m Q)^\top (R_n K) = Q^\top R_m^\top R_n K$$

Rotation matrices have a special property. They are orthogonal, which means the transpose equals the inverse:

$$R_m^\top = R_m^{-1} = R_{-m}$$

So the expression becomes:

$$Q^\top R_{-m} R_n K$$

Rotations combine by adding their angles. Rotating by $-m$ and then by n is the same as rotating by $n - m$:

$$R_{-m} R_n = R_{n-m}$$

Putting it together:

$$Q_m \cdot K_n = Q^\top R_{n-m} K$$

The result depends only on R_{n-m} . The absolute positions m and n never appear on their own. Only their difference $n - m$ survives.

This is the same conclusion as the angle argument, now proven through the matrix algebra.

How's this better???

In sinusoidal encodings, we had the following expression after attention score calculation:

$$Q_m \cdot K_n = (W_q \cdot \text{embed}_m) \cdot (W_k \cdot \text{embed}_n) + (W_q \cdot \text{embed}_m) \cdot (W_k \cdot PE_n) + (W_q \cdot PE_m) \cdot (W_k \cdot \text{embed}_n) + (W_q \cdot PE_m) \cdot (W_k \cdot PE_n)$$

And we objected that:

- **Unstructured interaction:** Adding position creates cross terms involving content and absolute position.
- **No guaranteed relative-only score:** Sinusoidal encodings can express relative relationships, but learned Q/K projections need not preserve them.
- **Additive fusion:** Content and position enter the projections as a sum; there is no explicit separation.

While in RoPE, we have the following expression, for one 2D pair:

$$Q_m \cdot K_n = \|Q_m\| \|K_n\| \cos((\alpha - \beta) + (m - n)\theta)$$

How is RoPE actually better than sinusoidal encodings?

Mixing content and position is not itself the problem.

- Attention needs both.
- **The improvement is that RoPE makes the positional dependence explicitly relative.**

RoPE still combines content and position, but guarantees that the explicit positional effect depends only on their relative separation.

How's this better???

Mixing content and position is not itself the problem. Attention needs both. The improvement is that RoPE makes the positional dependence explicitly relative.

With additive sinusoidal encodings, the cross terms compare:

- A token's content with the other token's **absolute position encoding**.
- Its **absolute position encoding** with the other token's content.
- Consequently, moving the same content from positions **5 and 8** to **105 and 108** can change the attention score, although the separation remains **3**.

With RoPE, for one 2D pair:

$$\text{score} = \|q\| \|k\| \cos \left(\underbrace{\alpha - \beta}_{\text{content angle}} + \underbrace{(m - n)\theta}_{\text{relative positional angle}} \right).$$

- Content and position still interact, but **absolute positions cancel**:

$$(5 - 8)\theta = (105 - 108)\theta = -3\theta.$$

- Holding the underlying query and key fixed, the score is therefore identical. This holds for every pair, and hence their sum.

How's this better???

The useful distinction is:

Additive sinusoidal encoding	RoPE
Content interacts with absolute position encodings.	Content interacts with relative rotations.
A common position shift can change the score.	A common position shift leaves the score unchanged, for fixed content vectors.

How's this better???

How is RoPE actually better than sinusoidal encodings?

Explicitly relative: Content and position still interact, but the positional angle depends only on $m - n$.

- **Same content + same separation $\rightarrow$ same score:** Moving tokens from positions **5 and 8** to **105 and 108** preserves the score, holding the underlying Q/K vectors fixed.
- **Preserves vector lengths:** Rotation changes Q/K directions without changing their magnitudes; position affects their alignment.
- **RoPE's advantage is not eliminating content–position interaction. It is giving that interaction a guaranteed relative-position structure.**

Scaling to Full Dimensions

So far we worked with a 2D query. Real queries have many dimensions, often 64 or 128 per head.

How does rotation work there?

The idea is simple. We split the vector into pairs and rotate each pair on its own.

Pairing the Dimensions

- Take a query of dimension d . Split it into $d/2$ consecutive pairs:

$$(q_0, q_1), (q_2, q_3), \dots, (q_{d-2}, q_{d-1})$$

- Each pair is a little 2D vector. We rotate each one exactly the way we did before.
- The key detail is that each pair gets its own frequency. Pair i uses frequency:

$$\theta_i = \frac{1}{10000^{2i/d}}$$

- This is the same frequency formula from sinusoidal encoding. Early pairs get high frequencies and rotate fast. Later pairs get low frequencies and rotate slowly.
- Each pair is rotated independently. The first pair does not interact with the second pair. There is no mixing across pairs.

The Block Diagonal Matrix

If we write the full rotation as one big $d \times d$ matrix.

It is block diagonal matrix. Along the diagonal sit $d/2$ small 2×2 rotation blocks, one per pair. Everywhere off the diagonal, the entries are zero.

$$R(m) = \begin{bmatrix} R(m\theta_0) & 0 & \cdots & 0 \\ 0 & R(m\theta_1) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & R(m\theta_{d/2-1}) \end{bmatrix}$$

- Each block $R(m\theta_i)$ is the familiar 2×2 rotation matrix for pair i at position m .

In this **block matrix**: each $R(m\theta_i)$ symbol represents an entire 2×2 matrix, and each 0 represents a 2×2 zero block.

- For $d = 6$, there are $d/2 = 3$ blocks:

$$R(m) = \begin{bmatrix} c_0 & -s_0 & 0 & 0 & 0 & 0 \\ s_0 & c_0 & 0 & 0 & 0 & 0 \\ 0 & 0 & c_1 & -s_1 & 0 & 0 \\ 0 & 0 & s_1 & c_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & c_2 & -s_2 \\ 0 & 0 & 0 & 0 & s_2 & c_2 \end{bmatrix}$$

- where

$$c_i = \cos(m\theta_i), s_i = \sin(m\theta_i)$$

- The three marked 2×2 diagonal regions are the three rotation blocks.

The Block Diagonal Matrix

For a six-dimensional query vector q , the multiplication is:

$$q^{(m)} = R(m)q = \begin{bmatrix} c_0 & -s_0 & 0 & 0 & 0 & 0 \\ s_0 & c_0 & 0 & 0 & 0 & 0 \\ 0 & 0 & c_1 & -s_1 & 0 & 0 \\ 0 & 0 & s_1 & c_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & c_2 & -s_2 \\ 0 & 0 & 0 & 0 & s_2 & c_2 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \end{bmatrix} = \begin{bmatrix} c_0 q_0 - s_0 q_1 \\ s_0 q_0 + c_0 q_1 \\ c_1 q_2 - s_1 q_3 \\ s_1 q_2 + c_1 q_3 \\ c_2 q_4 - s_2 q_5 \\ s_2 q_4 + c_2 q_5 \end{bmatrix}$$

Here, $c_i = \cos(m\omega_i)$ and $s_i = \sin(m\omega_i)$.

Each pair rotates independently at its own frequency:

- (q_0, q_1) rotates by $m\omega_0$.
- (q_2, q_3) rotates by $m\omega_1$.
- (q_4, q_5) rotates by $m\omega_2$.

The result is **one six-dimensional query vector**. The pairs stay in their respective slots; they are not added together.

Questions: Does Rotation Corrupt Semantic Meaning?

There is a natural objection to all of this.

- The direction of Q encodes the meaning of the token.
 - That is what we said.
 - The angle α carries semantic content.
- Rotation changes the direction.
 - After rotation, Q points in a new direction $\alpha + m\theta$.
- So, if direction is meaning, and rotation changes direction, then rotation must change meaning.
 - Rotation should corrupt the semantic content of the token.
- This reasoning has a flaw...

Does Rotation Corrupt Semantic Meaning?

The mistake is in the first step: **Meaning in attention is not the direction of a single vector.**

- What the attention mechanism actually computes is the dot product between a query and a key. And the dot product depends on the angle **between** them, not on either direction alone:

$$Q \cdot K = \cos(\alpha - \beta)$$

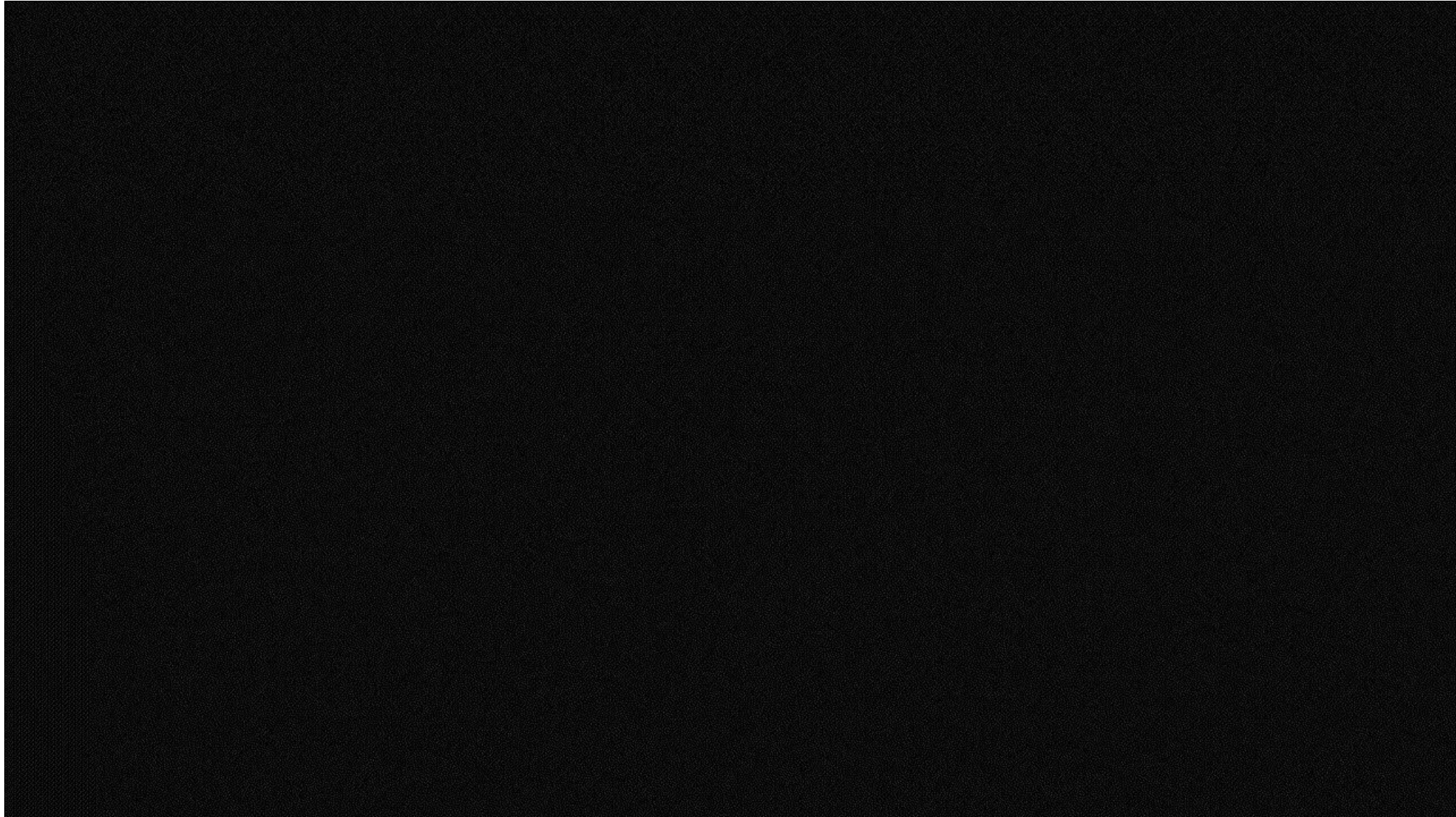
The semantic relationship is $\alpha - \beta$. It is a relationship between two vectors, not a property of one.

- Lets look again at what rotation does:

$$Q_m \cdot K_n = \cos((\alpha - \beta) + (m - n)\theta)$$

The semantic relationship is still inside the cosine. The rotation only added a positional piece next to it.

- The meaning is preserved. It is combined with position, not removed or destroyed.



Let's look at the Edges

Consider two tokens at the same position, so $m = n$.

The positional piece becomes $(m - n)\theta = 0$. The dot product is:

$$Q_m \cdot K_n = \cos(\alpha - \beta + 0) = \cos(\alpha - \beta)$$

This is exactly the original dot product, with no rotation effect at all.

Two tokens at the same position have their full semantic relationship, completely intact.

Rotation changed nothing about the meaning.

Rotation Preserves Length

There is another way to see that rotation does not damage the vectors.

- Rotation preserves length. A rotated vector has the same magnitude as the original. We can prove this directly.
- The squared length of a rotated vector is:

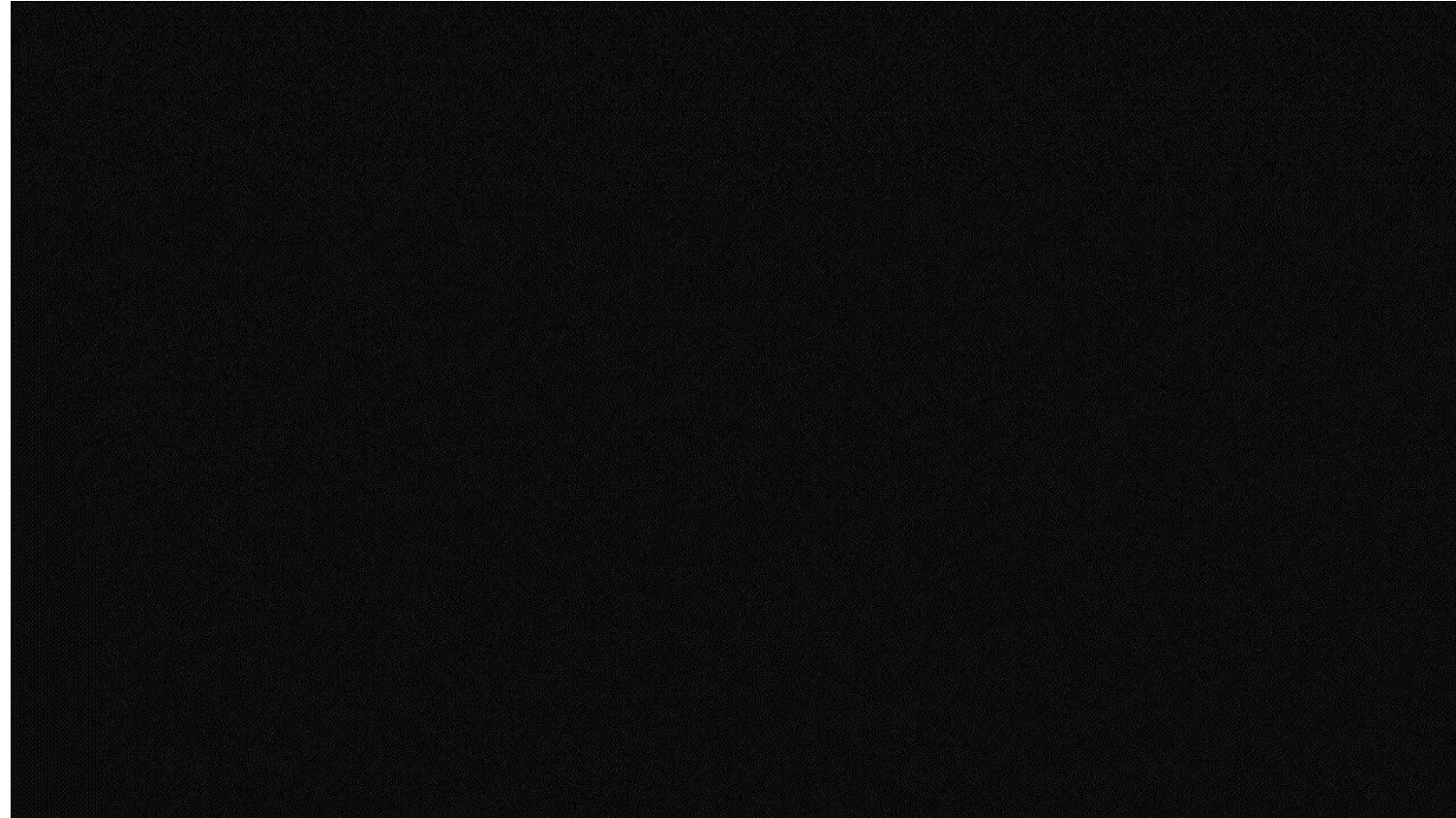
$$|Rv|^2 = (Rv)^\top (Rv) = v^\top R^\top R v$$

Since rotation is orthogonal, $R^\top R = I$:

$$v^\top R^\top R v = v^\top v = |v|^2$$

The length is unchanged.

The magnitude of a query or key often carries learned information too, and rotation leaves it completely unchanged. Only the direction turns.



The Angles α and β

The query angle α and the key angle β are not the embedding angles

- The angle α is not the angle of the token embedding.
- Remember the order of operations.
 - First the embedding goes through W_q .
 - This produces the query, Q . The angle α is the direction of that query, after the projection.

$$Q = W_q \cdot \text{embed}, \quad \alpha = \text{direction of } Q$$

- Same for the key. The angle β is the direction of $K = W_k \cdot \text{embed}$, after projection by W_k .
- These angles live in the query and key space. They are not angles in the original embedding space.

Meaning Carried by $(\alpha - \beta)$

If α comes out of a projection, why does the angle $(\alpha - \beta)$ carry semantic meaning at all?

- Because of training.

W_q and W_k are not fixed.

- They are learned through gradient descent, together with the rest of the model.
- During training, the loss pushes these matrices to arrange the queries and keys in a useful way.

The arrangement that minimizes the loss is the one where:

- Tokens that **should** attend to each other get a small angle $(\alpha - \beta)$. A small angle gives a high cosine, which gives a high attention score.
- Tokens that **should not** attend get a large angle $(\alpha - \beta)$. A large angle gives a low cosine, which gives a low attention score.

So the semantic meaning inside $(\alpha - \beta)$ is not inherited from the embedding space.

- It is learned.
- Training shapes W_q and W_k until the angle between a query and a key reflects how strongly the two tokens should attend.
- The query and key space is built by training to make $(\alpha - \beta)$ mean something.
 - RoPE then rotates within that learned space.

Limitations of RoPE: Aliasing

- Cosine repeats every 360 degrees, or 2π radians. So two different gaps can produce the exact same cosine.
- For a pair with frequency θ_i , two gaps g_1 and g_2 give the same value when their difference completes a full number of turns:

$$(g_1 - g_2) \cdot \theta_i = 2\pi \cdot k$$

for some whole number k . When this happens, the two gaps are indistinguishable in that pair. This is called aliasing.

Limitations of RoPE: Aliasing

Aliasing in Each Band

- Fast pairs alias quickly. With frequency near 1, the gap repeats about every 6 tokens. Gap 1 and gap 7 look almost the same to a fast pair.
 - This may not seem good, but it is fine. Fast pairs are only meant for short range precision. They do their job locally, and we never care about them for long distances.
- Slow pairs alias very slowly. With a frequency near 0.0001 (remember $\omega \propto 1/10,000$), they do not repeat until about 62,832 (i.e. $2\pi \cdot 10,000$) tokens. Within any normal context, they never alias.
 - So each band repeats at its own distance. Fast pairs repeat every few tokens. Slow pairs repeat only after tens of thousands.

Limitations of RoPE: Aliasing

Why the Spectrum Saves Us

- Here is the beautiful part. A single pair aliases often. But the full set of pairs almost never aliases all at once.
- For two gaps to be truly indistinguishable, they would have to alias in every pair at the same time. That means their difference would have to be a whole number of turns for the fast period, the medium period, and the slow period, all together.
- So even though every pair aliases on its own, the combination of all pairs gives each gap a unique fingerprint.

RoPE can calculate positions it has never learned to use

RoPE rotates each query/key pair by an angle proportional to position:

$$\text{angle} = m\omega_i$$

Attention depends on the **relative rotation** between positions:

$$\Delta\theta_i = (m - n)\omega_i$$

A model trained on 4,000-token sequences has learned how these rotations behave only over that range of distances.

- At longer distances, the formula still works, but the learned attention patterns may fail.
- **Being able to encode a position does not guarantee being able to use it correctly.**

A slow pair can change a behavior the model relied on

- Consider one pair with matching content.
- Its contribution to the attention score is $\cos(\Delta\theta)$
- Suppose this slow pair covers only **0° – 30°** across training distances.

Distance	Relative angle	Score contribution
Maximum training distance	30°	+0.866
Four times that distance	120°	−0.5

- During training, this pair consistently supports the matching content.
- At a much larger distance, **the same content now contributes negatively**, as cosine switches sign between 90° and 270° degrees.
- The model may have learned to rely on a stability that disappears outside training.

But don't fast pairs change sign too?

Yes. The difference is what the model experienced during training.

- **Fast pairs:** complete many rotations during training. Their sign changes are familiar.
- **Sufficiently slow pairs:** explore only a small arc. Their behavior over the rest of the circle is unfamiliar.
- At longer distances, the model also encounters unfamiliar **combinations of rotations across pairs.**

The problem is extrapolating learned behavior beyond the distances on which it was learned.

There is no universal “breaking angle.” A negative cosine alone does not imply failure, and one pair wrapping around does not make the entire encoding repeat.

Longer context also means more competition

Even if positional behavior remained reliable, attention must distribute its probability across more tokens.

Suppose one relevant token has score g , while N other tokens each have score 0.

$$P(\text{relevant token}) = \frac{e^{z_j}}{\sum_k e^{z_k}} = \frac{e^g}{e^g + N}$$

For $g = 2.5$, $e^{2.5} = 12.18249$, and N irrelevant tokens with score 0, $e^0 = 1$,

- **3 other tokens:** the relevant token receives **80.2%**.
- **31 other tokens:** it receives **28.2%**.

The content match stayed the same; its share of attention shrank.

This is a general long-context challenge that RoPE alone does not solve.

The formula

It comes directly from **softmax**, which converts attention scores into weights:

$$P(j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

In our simplified example:

- One relevant token has score g , contributing e^g .
- Each of the N other tokens has score 0, contributing $e^0 = 1$.

Therefore:

$$P(\text{relevant token}) = \frac{e^g}{e^g + \underbrace{e^0 + \dots + e^0}_{N \text{ other tokens}}} = \boxed{\frac{e^g}{e^g + N}}$$

Adding more competing tokens enlarges the denominator, even if the relevant token's score stays unchanged.

Here, P is the token's **attention weight**, not the probability that it is actually relevant.

Example

Suppose the relevant token’s attention score is **2**, and every other token’s score is **0**.

Since $e^2 \approx 7.39$, and $e^0 = 1$:

$$P(\text{relevant token}) = \frac{7.39}{7.39 + N}$$

The relevant token still scores higher than every competitor, but collectively they take a larger share of attention.

Other tokens N	Total tokens	Relevant token’s attention weight
3	4	$\frac{7.39}{7.39 + 3} = 71.1\%$
9	10	$\frac{7.39}{7.39 + 9} = 45.1\%$
99	100	$\frac{7.39}{7.39 + 99} = 6.9\%$

What must a context-extension method fix?

The goal: extend positional range while preserving useful distinctions and effective attention.

Problem	Direction of improvement
Longer distances produce unfamiliar rotations	Position interpolation: compress positional coordinates into the familiar range.
Uniform compression also shrinks nearby positional distinctions	Frequency-dependent scaling: preserve more fast-frequency detail while slowing other pairs.
Attention concentration also changes with context length	YaRN: combine selective frequency scaling with attention-score scaling.

These adjustments can help, but reliable extension may still require additional training.

[Position interpolation](#) · [YaRN](#)